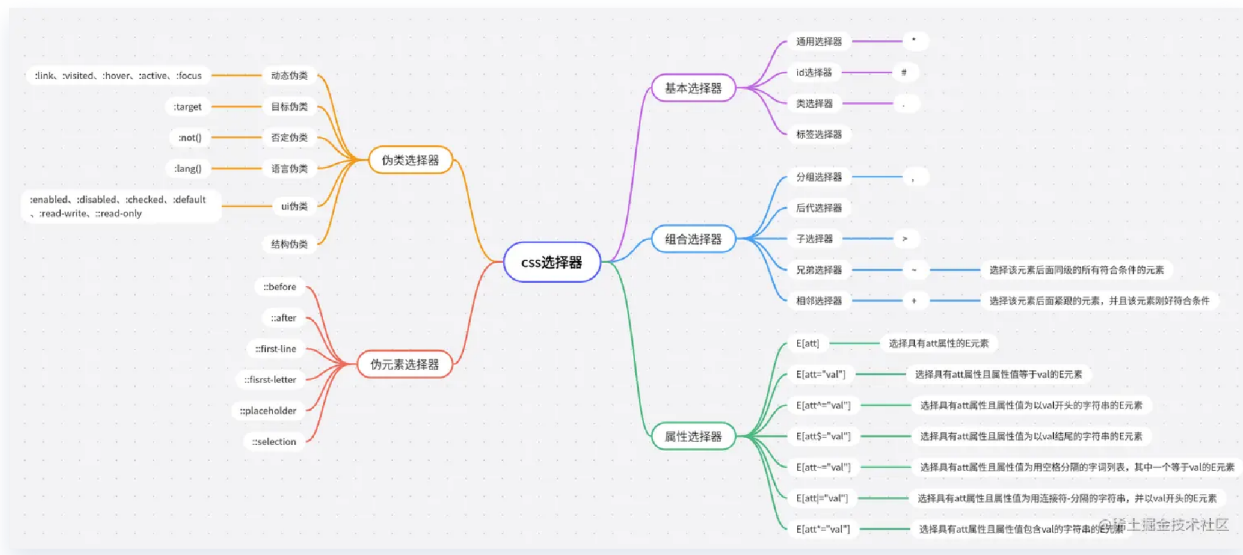


一.选择器



组合选择器

1. 后代选择器（Descendant Selector）：用空格分隔两个选择器，选择作为另一个元素后代的元素。例如：

```
div p {  
    color: red;  
}
```

上面的例子选择所有 `<p>` 元素，其父元素是 `<div>`。

2. 子元素选择器（Child Selector）：用 `>` 分隔两个选择器，选择作为另一个元素直接子元素的元素。例如：

```
ul > li {  
    list-style-type: none;  
}
```

上面的例子选择所有直接作为 `<ul>` 子元素的 `<li>` 元素。

3. 相邻兄弟选择器（Adjacent Sibling Selector）：使用 `+` 选择紧接在另一个元素后的第一个同级元素。例如：

```
h2 + p {  
    font-weight: bold;  
}
```

上面的例子选择紧接在 `<h2>` 元素后的第一个 `<p>` 元素，并将其字体加粗。

4. 通用兄弟选择器（General Sibling Selector）：使用 `~` 选择在另一个元素之后的所有同级元素。例如：

```
h2 ~ p {  
    color: blue;  
}
```

上面的例子选择紧跟在 `<h2>` 元素后面的所有 `<p>` 元素，并将它们的颜色设为蓝色。

伪类选择器

用于选择元素的特定状态，例如鼠标悬停、被点击等。常见的伪类选择器包括：

1. `:hover`：选择鼠标悬停在元素上时的样式。例如：

```
a:hover {  
    color: red;  
}
```

上面的例子表示当鼠标悬停在链接 `<a>` 元素上时，将文字颜色设为红色。

2. `:active`：选择被激活（被点击）的元素。例如：

```
button:active {  
    background-color: gray;  
}
```

上面的例子表示当按钮 `<button>` 被点击时，背景颜色变为灰色。

3. `:focus`：选择获得焦点的元素，通常用于表单元素。例如：

```
input:focus {  
    border: 2px solid blue;  
}
```

上面的例子表示当输入框 `<input>` 获得焦点时，边框变为蓝色。

4. `:first-child`：选择作为父元素的第一个子元素的元素。例如：

```
li:first-child {  
    font-weight: bold;  
}
```

上面的例子选择每个 `<li>` 元素，但只有它们是其父元素的第一个子元素时，才会将字体加粗。

5. `:nth-child(n)`：选择作为父元素的第 n 个子元素的元素。例如：

```
tr:nth-child(odd) {  
    background-color: #f2f2f2;  
}
```

上面的例子选择表格 `<tr>` 元素中奇数行，并将它们的背景颜色设为浅灰色。

伪元素选择器

伪元素选择器用于向文档中的特定部分添加样式，例如在元素的内容前面或后面插入一些内容。常见的伪元素选择器包括：

1. `::before`：在元素内容之前插入内容。例如：

```
p::before {
  content: ">> ";
  color: blue;
}
```

上面的例子表示在每个段落 `<p>` 的内容之前插入 ">> "，并将其颜色设为蓝色。

2. `::after`：在元素内容之后插入内容。例如：

```
button::after {
  content: " (click me)";
  color: red;
}
```

上面的例子表示在每个按钮 `<button>` 的内容之后插入 " (click me)", 并将其颜色设为红色。

3. `::first-line`：选择元素的第一行文本。例如：

```
p::first-line {  
    font-weight: bold;  
}
```

上面的例子表示将每个段落 `<p>` 的第一行文本加粗显示。

4. `::selection`：选择用户选中的文本部分。例如：

```
::selection {  
    background-color: yellow;  
    color: black;  
}
```

上面的例子表示当用户选中文本时，将选中部分的背景颜色设为黄色，文字颜色设为黑色。

5. `::placeholder`：选择表单元素的占位符文本。例如：

```
input::placeholder {  
    color: gray;  
}
```

上面的例子表示将输入框 `<input>` 的占位符文本颜色设为灰色。

二.选择器的权重

只有在 `css` 冲突的时候才会用到样式权重的计算，权重大的会覆盖权重小的样式，当权重值一样的时候，后面的样式会覆盖前面的样式。

权重值到底怎么计算呢？下面我来说明一下。

1. `!important` 会覆盖页面内任何位置的元素样式,权重值为无穷大

2. 内联样式，即写在标签里面的 `style` 样式，权重值为 `1000`。
3. `id` 选择器，权重值为 `0100`。
4. 类选择器、伪类选择器、属性选择器，权重值为 `0010`。
5. 标签选择器、伪元素选择器，权重值为 `0001`。
6. 组合选择器、通配符选择器，权重值为 `0000`。

我们在比较选择器的权重时，会对选择器的权重进行相加，但是注意：相加不存在进位，比如当有11个标签选择器和1个类选择器同时选中同一元素时，仍然是这个类选择的权重高。

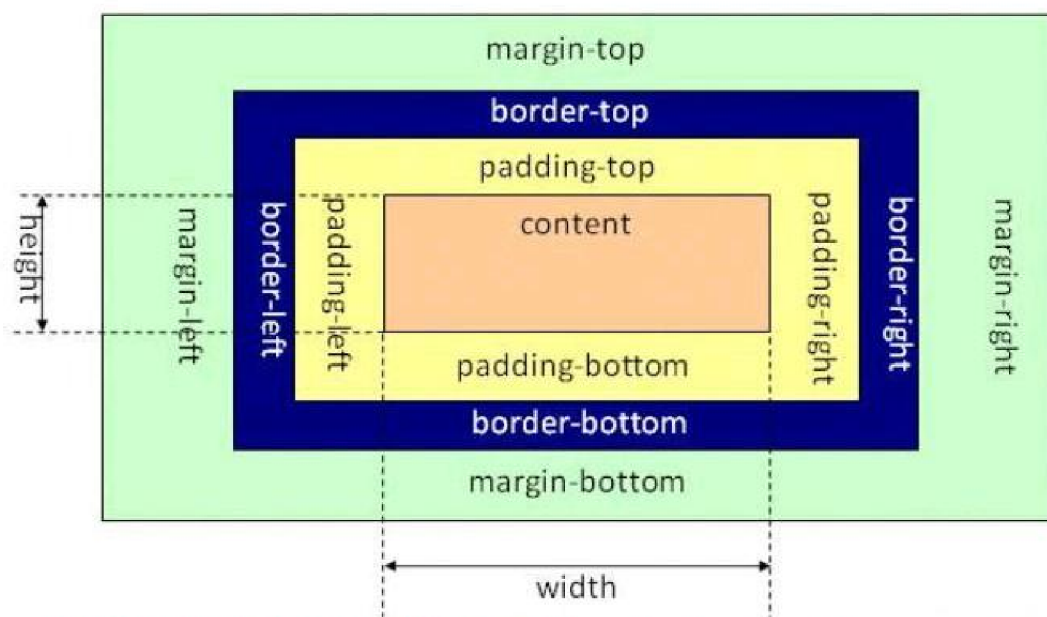
如果权重相同，后面的选择器覆盖前面的选择器

如果权重不同，选择器的权重大的覆盖较小的

三. CSS的盒模型

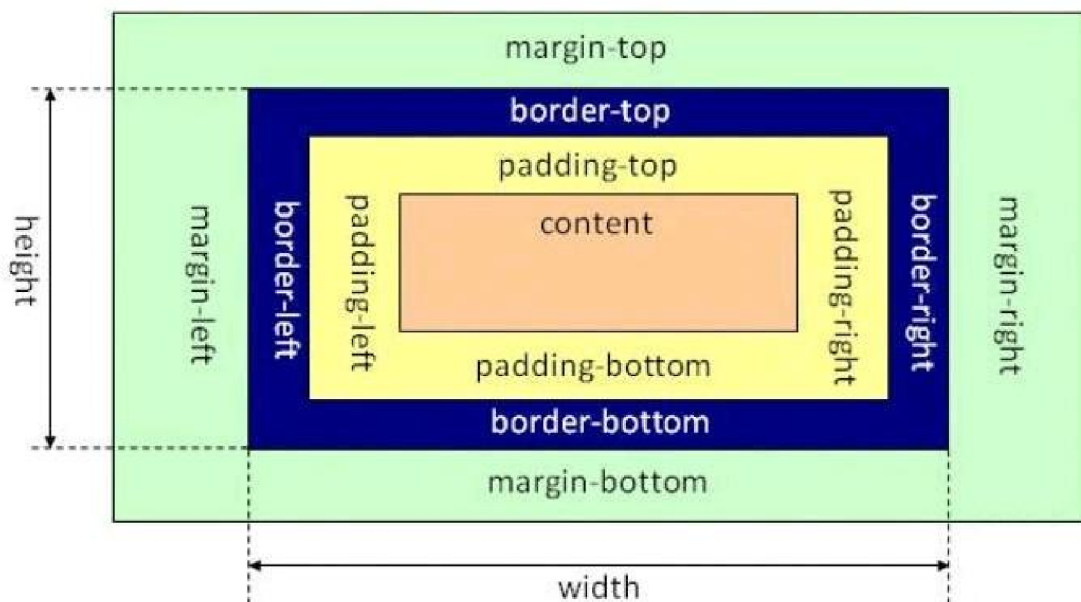
CSS盒模型本质上是一个盒子，封装周围的HTML元素，它包括：外边距（margin）、边框（border）、内边距（padding）、实际内容（content）四个属性。

■ 标准盒子模型



从上图可以看到标准 W3C 盒子模型的范围包括 margin、border、padding、content，并且 content 部分不包含其他部分

■ IE 盒子模型



从上图可以看到 IE 盒子模型的范围也包括 margin、border、padding、content，和标准 W3C 盒子模型不同的是：IE 盒子模型的 content 部分包含了 border 和 padding

标准盒模型（Standard Box Model）是CSS的默认盒模型。在标准盒模型中，一个元素的总宽度（width）和总高度（height）等于其内容区域的宽度（content width）和高度（content height）。padding、border和margin都会被添加到元素的总宽度和总高度之外。

怪异盒模型（Quirks Box Model）是在某些旧版本的浏览器中存在的盒模型。在怪异盒模型中，一个元素的总宽度和总高度包括了其内容区域、内边距和边框的宽度和高度，但不包括外边距的宽度和高度。换句话说，怪异盒模型中的宽度和高度不再表示内容区域的大小，而是包括了内边距和边框。

标准盒模型和怪异盒模型之间的转换：`box-sizing :content-box/border-box/inherit ;`

当为content-box时，将采取标准模式进行解析计算；

当为border-box时，将采取怪异模式解析计算；

当为inherit时，将从父元素来继承box-sizing属性的值；

补充：margin负值问题

margin 负值问题

- ◆ margin-top 和 margin-left 负值，元素向上、向左移动
- ◆ margin-right 负值，右侧元素左移，自身不受影响
- ◆ margin-bottom 负值，下方元素上移，自身不受影响

@11352116

四.margin合并（也叫margin塌陷）及其解决方案

什么是margin合并？

发生在文档标准流中的问题，它是指相邻的两个元素的垂直方向上的margin发生重叠的现象，比如父子元素的margin-top并不会相加而是会重叠，它们共同使用了二者中margin较大的外边距，又比如两个兄弟元素的margin-bottom和margin-top并不会相加，而是会重叠。

如何解决margin合并？

在解决父子元素的margin合并时，我们通常有两种解决办法：

- 1.给父元素设置外边框（border）或者内边距（padding），但是不建议这种做法，因为这样可能会让盒子变大，进而对我们总体的布局造成影响
- 2.将父元素设为 BFC，这是一种推荐的做法。创建 BFC 的方式包括给父元素设置 overflow 属性为除 visible 以外的值、float 属性不为 none、使用 display: inline-block

在解决兄弟元素之间margin合并时，我们通常的解决方案也有两种：

- 1.只设置其中一个元素的margin值即可（推荐的做法）；
- 2.给每一个元素添加父元素，然后触发BFC

第二种是不推荐的做法，因为会改变HTML的结构

五.消除浮动

先来说说float布局

浮动会脱标（脱离标准流）

当一个元素浮动之后，它会被移出正常的文档流，然后向左或者向右平移，一直平移直到碰到了所处的容器的边框，或者碰到另外一个浮动的元素

为什么要清除？

我们很多时候不方便给父盒子的高度，因为我们不消除有多少子盒子，有多少内容。经常的做法会让内容撑开父盒子的高度。但是如果父盒子中有子盒子浮动之后，就会影响到后面的盒子，由于浮动元素会脱离文档流，所以导致不占据页面空间，如果一个元素中包含的元素全部是浮动元素，那么该元素高度将变成0，也就是我们常说的高度塌陷。并且浮动的盒子还会在其后的标准流的盒子覆盖，解决这个问题的方法就要清除浮动带来的影响

清除浮动其实叫做 **闭合浮动** 更合适，~~因为是把浮动的元素圈起来，让父元素闭合出口和入口不让他们出来影响其他的元素。~~在 CSS 中，clear 属性用于清除浮动，其基本语法格式如下：

```
选择器 { clear : 属性值 ; }  
/*属性值为left,清除左侧浮动的影响  
   属性值为right,清除右侧浮动的影响  
   属性值为both,同时清除左右两侧浮动的影响*/
```

清除的方法：

1. 额外标签

通过在浮动元素的末尾添加一个空的标签，在这个标签上使用clear属性，既可以写成内联样式，也可以写成外部样式，但是这样做添加了无意义的标签，结构化比较差，所以不推荐使用。

2. overflow——触发BFC

添加 `overflow` 样式方法。比如为了防止父元素的高度塌陷，我们给父元素触发BFC。为了防止一个非浮动的盒子被浮动的盒子覆盖，我们给非浮动的盒子触发BFC。

这种方法代码比较简洁且兼容性好，可以通过触发 `BFC` 方式，但是因为本身 `overflow` 的本质是 溢出隐藏 的效果，所以有的时候也会有一些问题存在，比如内容增多的时候不会自动换行导致内容被隐藏掉，无法显示出要溢出的元素。

3.伪元素法（最常用）

这种方法符合闭合浮动思想，结构语义化正确。

使用after伪元素清除浮动

给父元素添加一个类名为clearfix,然后对这个类名在css里使用after伪元素选择器，目的是为了在浮动元素后面添加一个空的块级元素，并设置其 `clear` 属性为 `both`

::after (:after) CSS 伪元素 用来创建一个伪元素，作为已选中元素的最后一个子元素

```
.clearfix:after{
    content:"";
    height:0;      /*盒子高度为0，看不见*/
    display:block;  /*插入伪元素是行内元素，要转化为块级元素*/
    clear:both;
}

.clearfix {
    *zoom: 1;    /* *只有IE6,7识别 */
}
```

使用before和after双伪元素清除浮动（推荐）

这里是为了实现更好的兼容性，包括在这个例子中我们使用了display: table;而不是block，也是为了更好的兼容性

```
.clearfix:before, .clearfix:after {
    content: "";
    display: table; /* 这句话可以触发 BFC BFC 可以清除浮动 */
}
.clearfix:after {
    clear: both;
}
.clearfix {
    *zoom: 1;
}
```

六.css定位属性position（文档流）

在解释position的属性值之前，最好先说明一下什么是文档流。

将窗体自上而下分成一行行，并在每行中按从左至右的顺序排放元素，即为文档流(normal flow直译为常规流、正常流)。

每个非浮动块级元素都独占一行，浮动元素则按规定浮在行的一端。若当前行容不下，则另起新行再浮动。内联元素不会独占一行。几乎所有元素(包括块级，内联和列表元素)均可生成子行，用于摆放子元素。

有三种情况将使得元素脱离文档流而存在，分别是 浮动，绝对定位，固定定位。

position属性值有五个，分别是：

- static
- relative
- absolute
- fixed
- sticky

现在分别来说明各个属性的含义：

1.static

这个就是position默认的属性,设置为static相当于不设置position属性

2.relative

相对定位，所谓相对，即相对的是自己在文档流中的原始定位，并且不会脱离文档流，还是处于文档流之中。我们可以给它设置left,top,right,bottom，z-index值，让其相对与自己在文档流中的原始定位进行移动。

3. absolute

绝对定位，会脱离文档流，定位参照对象是最近一级拥有定位的祖先元素，可以通过left、right、top、bottom来进行位置调整；如果一直往上层元素找不到有定位的元素，那么最终的参照对象为文档。（所谓有定位的元素就是设置了position属性但是position属性值不为static的情况）

4. fixed

固定定位，这个属性会**脱离文档流**，给定一些定位参数，它就会相对于浏览器的窗口进行定位，由于是相对于浏览器的窗口进行定位，因此在页面发生滚动时，它也会有随着页面滚动而滚动的效果，这一切都是因为它相对于浏览器的窗口进行定位而并非是相对于文档定位

相对于文档，就是相对于整个页面来进行布局，而相对于窗口，则是相对于**浏览器的可视区域**进行定位，这二者有本质的区别的。

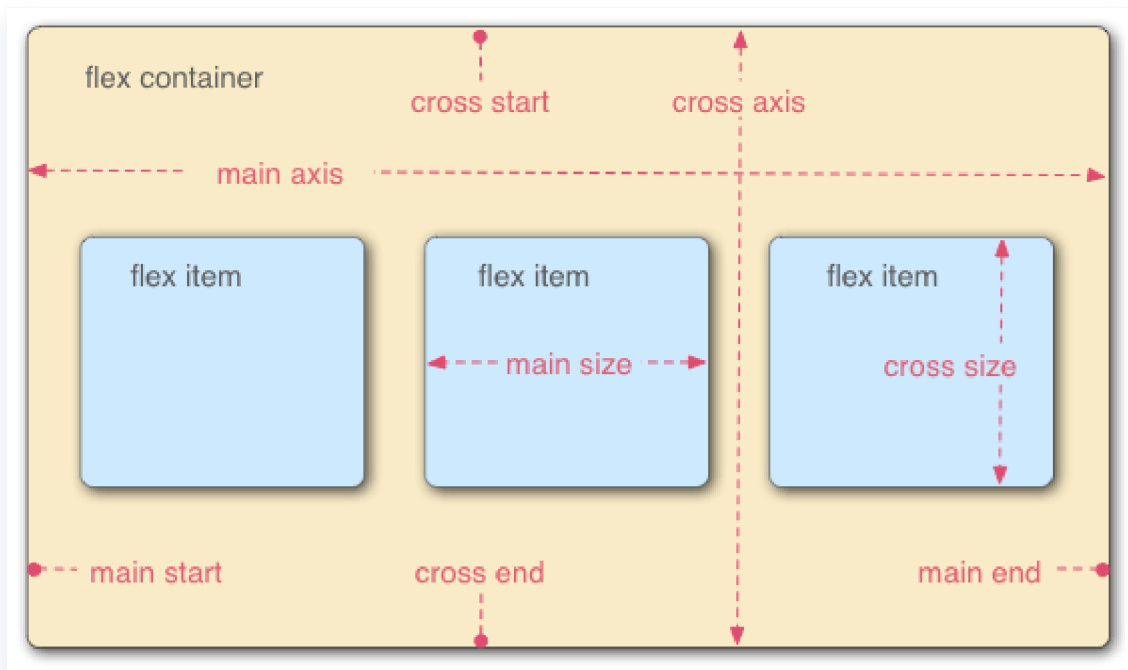
5.sticky

这个属性并非w3c推荐的标准，且兼容性不好。设置了sticky的元素，在屏幕范围（viewport）时该元素的位置并不受到定位影响（设置lrb等属性无效），当该元素的位置将要移出屏幕范围时，定位又会变成fixed，根据设置的lrb等属性成固定位置的效果。

七.flex布局

通常的做法是给一个父元素设置 `display: flex;`，此时这个父盒子可以被称之为flex容器（flex container），而它的子元素都叫做项目（item）

容器默认存在两根轴：水平的主轴（main axis）和垂直的交叉轴（cross axis）。项目默认沿主轴排列。



因此flex布局的属性就分为设置在容器上的和设置在项目上的

1.容器上的：

- **flex-direction:**

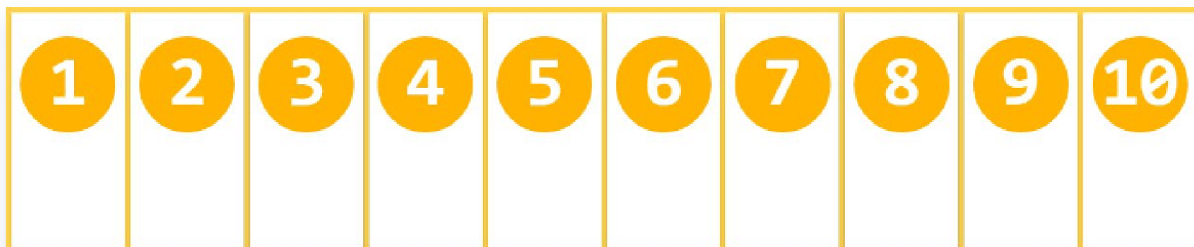
```
.box {  
  flex-direction: row | row-reverse | column | column-reverse;  
}
```

- row（默认值）：主轴为水平方向，起点在左端。
- row-reverse：主轴为水平方向，起点在右端。
- column：主轴为垂直方向，起点在上沿。
- column-reverse：主轴为垂直方向，起点在下沿。

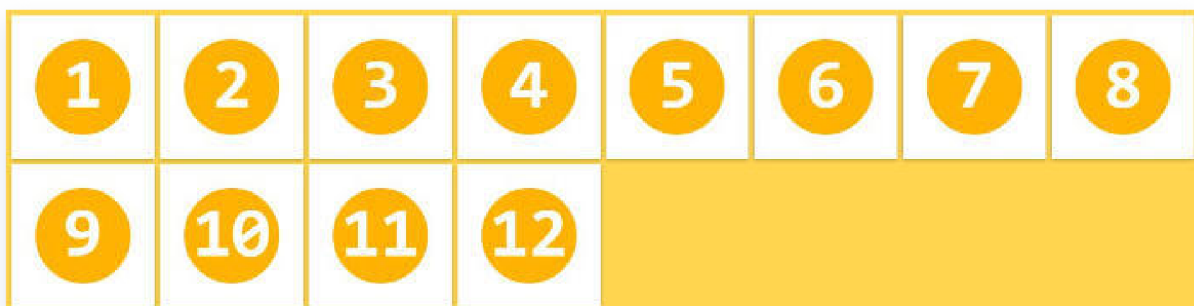
- **flex-wrap:**

```
.box{  
  flex-wrap: nowrap | wrap  
}
```

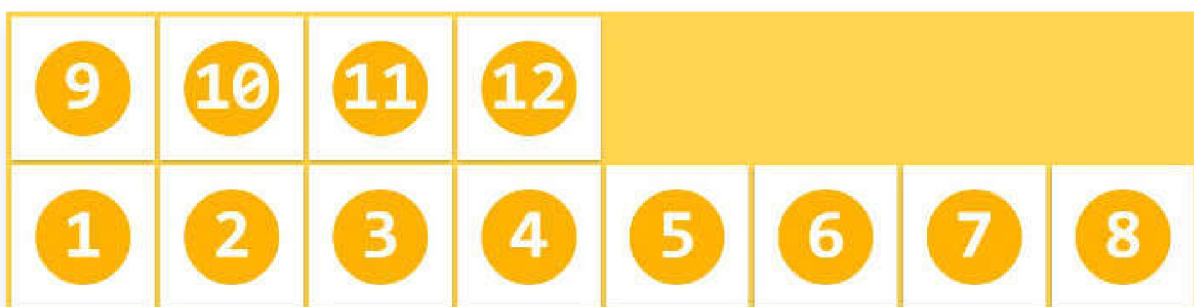
(1) nowrap (默认)：不换行。



(2) wrap: 换行，第一行在上方。



(3) wrap-reverse: 换行，第一行在下方。



- **flex-flow:**

flex-flow属性是flex-direction属性和flex-wrap属性的简写形式


```
.box {  
  flex-flow: <flex-direction> <flex-wrap>;  
}
```

eg:

```
.box {  
  flex-flow: column nowrap  
}
```

- **justify-content:**

justify-content属性定义了项目在主轴上的对齐方式。

```
.box {  
  justify-content: flex-start | flex-end | center | space-between | space-around;  
}
```

它可能取5个值，具体对齐方式与轴的方向有关。下面假设主轴为从左

- - flex-start（默认值）：左对齐
 - flex-end：右对齐
 - center：居中
 - space-between：两端对齐，项目之间的间隔都相等。

align-items:

align-items属性定义项目在交叉轴上如何对齐。

```
.box {  
  align-items: flex-start | flex-end | center | baseline |  
  stretch;  
}
```

flex-start: 交叉轴的起点对齐。

flex-end: 交叉轴的终点对齐。

center: 交叉轴的中点对齐。

baseline: 项目的第一行文字的基线对齐。

stretch (默认值): 如果项目未设置高度或设为auto, 将占满整个容器的高度。

- **align-content**属性:

该属性对单行弹性盒子模型无效。(即: 带有 `flex-wrap: nowrap`)

align-content 属性定义了项目在交叉轴之间的分布。

- **flex-start**: 与交叉轴的起点对齐。
- **flex-end**: 与交叉轴的终点对齐。
- **center**: 与交叉轴的中点对齐。
- **space-between**: 与交叉轴两端对齐, 轴线之间的间隔平均分布。
- **space-around**: 每根轴线两侧的间隔都相等。所以, 轴线之间的间隔比轴线与边框的间隔大一倍。
- **stretch** (默认值): 轴线占满整个交叉轴。

```
.box {  
  align-content: flex-start | flex-end | center | space-  
  between | space-around | stretch;  
}
```

2.项目上的:

- **order属性**

order属性定义项目的排列顺序。数值越小，排列越靠前，默认为0。

```
.item {  
    order: <integer>;  
}
```

- **flex-grow:**

flex-grow属性定义项目的放大比例，默认为0，即如果存在剩余空间，也不放大。

```
.item {  
    flex-grow: <number>; /* default 0 */  
}
```

如果所有项目的flex-grow属性都为1，则它们将等分剩余空间（如果有的话）。如果一个项目的flex-grow属性为2，其他项目都为1，则前者占据的剩余空间将比其他项多一倍。

- **flex-shrink:**

flex-shrink属性定义了项目的缩小比例，默认为1，即如果空间不足，该项目将缩小。（因为wrap属性默认的值是nowrap,这样规定使得默认的属性值有一致性，不会冲突）

```
.item {  
    flex-shrink: <number>; /* default 1 */  
}
```

如果所有项目的flex-shrink属性都为1，当空间不足时，都将等比例缩小。如果一个项目的flex-shrink属性为0，其他项目都为1，则空间不足时，前者不缩小。负值对该属性无效。

- **flex-basis:**

flex-basis属性定义项目占据的主轴空间（main size）。浏览器会根据这个属性，计算主轴是否有多余空间。它的默认值为auto，即项目的本来大小。

```
.item {  
  flex-basis: <length> | auto; /* default auto */  
}
```

它可以设为跟width或height属性一样的值（比如350px），则项目将占据固定空间。

- **flex:**

flex属性是flex-grow, flex-shrink 和 flex-basis的简写，默认值为0 1 auto。后两个属性可选。

```
.item {  
  flex: none | [ <'flex-grow'> <'flex-shrink'>? ||  
    <'flex-basis'> ]  
}
```

建议优先使用这个属性而不是分开写三个独立的属性

- **align-self:**

align-self属性允许单个项目有与其他项目不一样的对齐方式，可覆盖align-items属性。默认值为auto，表示继承父元素的align-items属性，如果没有父元素，则等同于stretch。

```
.item {  
    align-self: auto | flex-start | flex-end | center |  
    baseline | stretch;  
}
```

八.Grid布局

和flex布局的区别：

Flex 布局是轴线布局，只能指定"项目"针对轴线的位置，可以看作是一维布局。Grid 布局则是将容器划分成"行"和"列"，产生单元格，然后指定"项目所在"的单元格，可以看作是二维布局。Grid 布局远比 Flex 布局强大。

基本概念：

也有容器和项目的概念，与flex布局基本一致，一个不同的点是：项目只能是容器的顶层子元素，不包含项目的子元素，Grid 布局只对项目生效。

容器里面的水平区域称为"行"（row），垂直区域称为"列"（column）。行和列的交叉区域，称为"单元格"（cell）。

正常情况下， n 行有 $n + 1$ 根水平网格线， m 列有 $m + 1$ 根垂直网格线，比如三行就有四根水平网格线。

Grid 布局的属性分成两类。一类定义在容器上面，称为容器属性；另一类定义在项目上面，称为项目属性。

容器属性

1 display 属性

`display: grid` 指定一个容器采用网格布局。

```
div {  
  display: grid;  
}
```

默认情况下，容器元素都是块级元素，但也可以设成行内元素。

```
div {  
  display: inline-grid;  
}
```

上面代码指定 `div` 是一个行内元素，该元素内部采用网格布局。

注意，设为网格布局以后，容器子元素（项目）的 `float`、`display: inline-block`、`display: table-cell`、`vertical-align` 和 `column-*` 等设置都将失效。

2 grid-template-columns 属性 , grid-template-rows 属性

容器指定了网格布局以后，接着就要划分行和列。`grid-template-columns` 属性定义每一列的列宽，`grid-template-rows` 属性定义每一行的行高。

```
.container {  
  display: grid;  
  grid-template-columns: 100px 100px 100px;  
  grid-template-rows: 100px 100px 100px;  
}
```

上面代码指定了一个三行三列的网格，列宽和行高都是 `100px`。

除了使用绝对单位，也可以使用百分比。

```
.container {  
  display: grid;  
  grid-template-columns: 33.3% 33.3% 33.3%;  
  grid-template-rows: 33.3% 33.3% 33.3%;  
}
```

(1) repeat()

有时候，重复写同样的值非常麻烦，尤其网格很多时。这时，可以使用 `repeat()` 函数，简化重复的值。上面的代码用 `repeat()` 改写如下。

```
.container {  
  display: grid;  
  grid-template-columns: repeat(3, 33.3%);  
  grid-template-rows: repeat(3, 33.3%);  
}
```

`repeat()` 接受两个参数，第一个参数是重复的次数（上例是3），第二个参数是所要重复的值。

`repeat()` 重复某种模式也是可以的。

```
grid-template-columns: repeat(2, 100px 20px 80px);/*  
（也就是100px,200px,80px）这个列构成的序列重复两次*/
```

上面代码定义了6列，第一列和第四列的宽度为 `100px`，第二列和第五列为 `20px`，第三列和第六列为 `80px`。

(2) `auto-fill` 关键字

有时，单元格的大小是固定的，但是容器的大小不确定。如果希望每一行（或每一列）容纳尽可能多的单元格，这时可以使用 `auto-fill` 关键字表示自动填充。

```
.container {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, 100px);  
}
```


上面代码表示每列宽度 `100px`，然后自动填充，直到容器不能放置更多的列。



除了 `auto-fill`，还有一个关键字 `auto-fit`，两者的行为基本是相同的。只有当容器足够宽，可以在一行容纳所有单元格，并且单元格宽度不固定的时候，才会有[行为差异](#)：`auto-fill` 会用空格子填满剩余宽度，`auto-fit` 则会尽量扩大单元格的宽度。

(3) `fr` 关键字

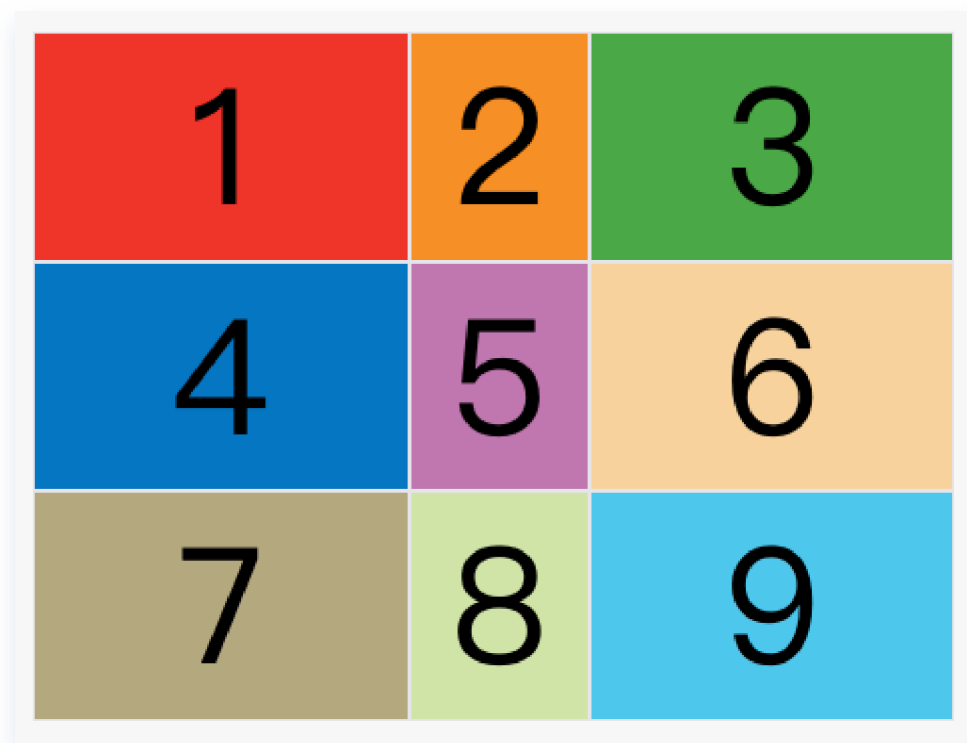
为了方便表示比例关系，网格布局提供了 `fr` 关键字（fraction 的缩写，意为"片段"）。如果两列的宽度分别为 `1fr` 和 `2fr`，就表示后者是前者的两倍。

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 1fr;  
}
```

`fr` 可以与绝对长度的单位结合使用，这时会非常方便。

```
.container {  
  display: grid;  
  grid-template-columns: 150px 1fr 2fr;  
}
```

[上面代码](#)表示，第一列的宽度为150像素，第二列的宽度是第三列的一半。



(4) minmax()

`minmax()` 函数产生一个长度范围，表示长度就在这个范围之内。它接受两个参数，分别为最小值和最大值。

```
grid-template-columns: 1fr 1fr minmax(100px, 1fr);
```

上面代码中，`minmax(100px, 1fr)` 表示列宽不小于 `100px`，不大于 `1fr`。

(5) `auto` 关键字

`auto` 关键字表示由浏览器自己决定长度。

```
grid-template-columns: 100px auto 100px;
```

上面代码中，第二列的宽度，基本上等于该列单元格的最大宽度，除非单元格内容设置了 `min-width`，且这个值大于最大宽度。

(6) 网格线的名称

`grid-template-columns` 属性和 `grid-template-rows` 属性里面，还可以使用方括号，指定每一根网格线的名字，方便以后的引用。

```
.container {  
  display: grid;  
  grid-template-columns: [c1] 100px [c2] 100px [c3] auto  
    [c4];  
  grid-template-rows: [r1] 100px [r2] 100px [r3] auto  
    [r4];  
}
```

上面代码指定网格布局为3行 x 3列，因此有4根垂直网格线和4根水平网格线。方括号里面依次是这八根线的名字。

网格布局允许同一根线有多个名字，比如 `[fifth-line row-5]`。

(7) 布局实例

`grid-template-columns` 属性对于网页布局非常有用。两栏式布局只需要一行代码。

```
.wrapper {  
  display: grid;  
  grid-template-columns: 70% 30%;  
}
```

上面代码将左边栏设为70%，右边栏设为30%。

传统的十二网格布局，写起来也很容易。

```
grid-template-columns: repeat(12, 1fr);
```

3 `grid-row-gap` 属性, `grid-column-gap` 属性, `grid-gap` 属性

`grid-row-gap` 属性设置行与行的间隔（行间距），`grid-column-gap` 属性设置列与列的间隔（列间距）

如果 `grid-gap` 省略了第二个值，浏览器认为第二个值等于第一个值。

根据最新标准，上面三个属性名的 `grid-` 前缀已经删除，`grid-column-gap` 和 `grid-row-gap` 写成 `column-gap` 和 `row-gap`，`grid-gap` 写成 `gap`。

4 grid-template-areas 属性

网格布局允许指定"区域"（area），一个区域由单个或多个单元格组成。 `grid-template-areas` 属性用于定义区域。

```
.container {  
  display: grid;  
  grid-template-columns: 100px 100px 100px;  
  grid-template-rows: 100px 100px 100px;  
  grid-template-areas: 'a b c'  
                      'd e f'  
                      'g h i';  
}
```

多个单元格合并成一个区域的写法如下。

```
grid-template-areas: 'a a a'  
                   'b b b'  
                   'c c c';
```

上面代码将9个单元格分成 `a`、`b`、`c` 三个区域。

下面是一个布局实例。

```
grid-template-areas: "header header header"
                    "main main sidebar"
                    "footer footer footer";
```

上面代码中，顶部是页眉区域 `header`，底部是页脚区域 `footer`，中间部分则为 `main` 和 `sidebar`。

如果某些区域不需要利用，则使用"点"（`.`）表示。

```
grid-template-areas: 'a . c'
                    'd . f'
                    'g . i';
```

上面代码中，中间一列为点，表示没有用到该单元格，或者该单元格不属于任何区域。

注意，区域的命名会影响到网格线。每个区域的起始网格线，会自动命名为 `区域名-start`，终止网格线自动命名为 `区域名-end`。

比如，区域名为 `header`，则起始位置的水平网格线和垂直网格线叫做 `header-start`，终止位置的水平网格线和垂直网格线叫做 `header-end`。

5 grid-auto-flow 属性

划分网格以后，容器的子元素会按照顺序，自动放置在每一个网格。默认的放置顺序是"先行后列"，即先填满第一行，再开始放入第二行

这个顺序由 `grid-auto-flow` 属性决定，默认值是 `row`，即"先行后列"。也可以将它设成 `column`，变成"先列后行"。

```
grid-auto-flow: column;
```

`grid-auto-flow` 属性除了设置成 `row` 和 `column`，还可以设成 `row dense` 和 `column dense`。这两个值主要用于，某些项目指定位置以后，剩下的项目怎么自动放置。

现在修改设置，设为 `row dense`，表示"先行后列"，并且尽可能紧密填满，尽量不出现空格。

```
grid-auto-flow: row dense;
```

如果将设置改为 `column dense`，表示"先列后行"，并且尽量填满空格。

```
grid-auto-flow: column dense;
```

6 justify-items 属性， align-items 属性， place-items 属性

`justify-items` 属性设置单元格内容的水平位置（左中右），`align-items` 属性设置单元格内容的垂直位置（上中下）。

```
.container {  
  justify-items: start | end | center | stretch;  
  align-items: start | end | center | stretch;  
}
```

这两个属性的写法完全相同，都可以取下面这些值。

- start：对齐单元格的起始边缘。
- end：对齐单元格的结束边缘。
- center：单元格内部居中。
- stretch：拉伸，占满单元格的整个宽度（默认值）。

```
.container {  
  justify-items: start;  
}
```

上面代码表示，单元格的内容左对齐，效果如下图。

```
.container {  
  align-items: start;  
}
```

`place-items` 属性是 `align-items` 属性和 `justify-items` 属性的合并简写形式。


```
place-items: <align-items> <justify-items>;
```

下面是一个例子。

```
place-items: start end;
```

如果省略第二个值，则浏览器认为与第一个值相等。

7 justify-content 属性， align-content 属性， place-content 属性

`justify-content` 属性是整个内容区域在容器里面的水平位置（左中右），`align-content` 属性是整个内容区域的垂直位置（上中下）。

```
.container {  
  justify-content: start | end | center | stretch |  
  space-around | space-between | space-evenly;  
  align-content: start | end | center | stretch | space-  
  around | space-between | space-evenly;  
}
```

这两个属性的写法完全相同，都可以取下面这些值。（下面的图都以 `justify-content` 属性为例，`align-content` 属性的图完全一样，只是将水平方向改成垂直方向。）

- start - 对齐容器的起始边框。

- end - 对齐容器的结束边框。

- center - 容器内部居中。

- stretch - 项目大小没有指定时，拉伸占据整个网格容器。

- space-around - 每个项目两侧的间隔相等。所以，项目之间的间隔比项目与容器边框的间隔大一倍。

- space-between - 项目与项目的间隔相等，项目与容器边框之间没有间隔。

- space-evenly - 项目与项目的间隔相等，项目与容器边框之间也是同样长度的间隔。

`place-content` 属性是 `align-content` 属性和 `justify-content` 属性的合并简写形式。

```
place-content: <align-content> <justify-content>
```

下面是一个例子。

```
place-content: space-around space-evenly;
```

如果省略第二个值，浏览器就会假定第二个值等于第一个值。

8 grid-auto-columns 属性， grid-auto-rows 属性

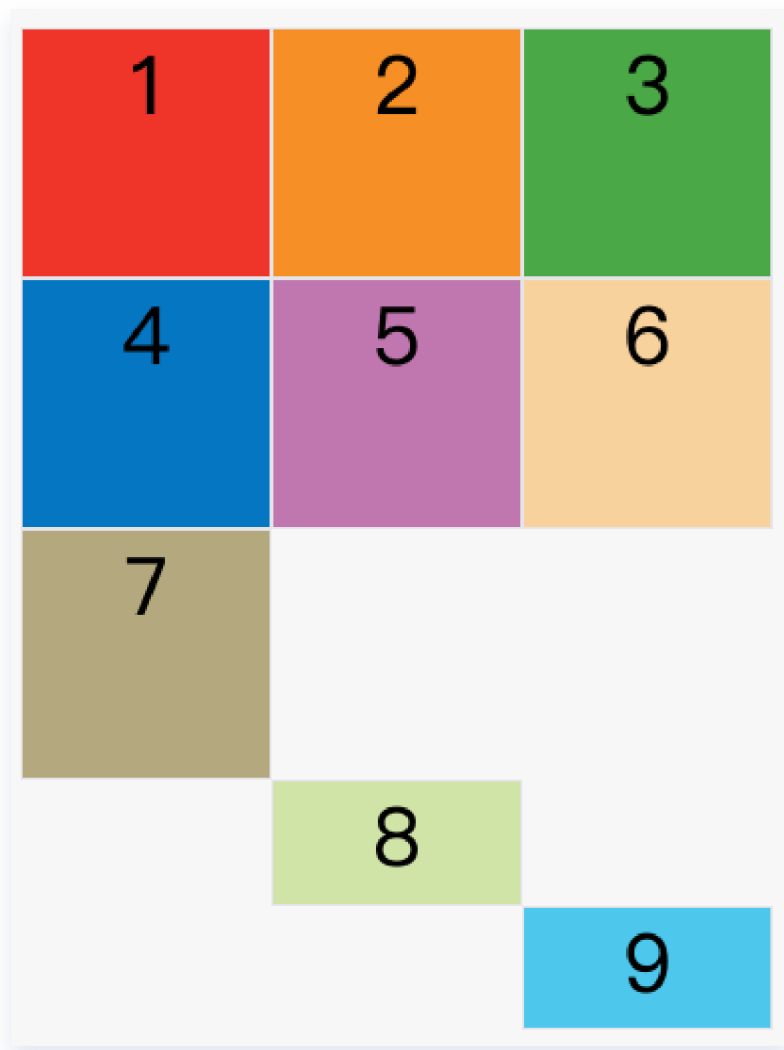
有时候，一些项目的指定位置，在现有网格的外部。比如网格只有3列，但是某一个项目指定在第5行。这时，浏览器会自动生成多余的网格，以便放置项目。

`grid-auto-columns` 属性和 `grid-auto-rows` 属性用来设置，浏览器自动创建的多余网格的列宽和行高。它们的写法与 `grid-template-columns` 和 `grid-template-rows` 完全相同。如果不指定这两个属性，浏览器完全根据单元格内容的大小，决定新增网格的列宽和行高。

[下面的例子](#)里面，划分好的网格是3行 x 3列，但是，8号项目指定在第4行，9号项目指定在第5行。

```
.container {  
  display: grid;  
  grid-template-columns: 100px 100px 100px;  
  grid-template-rows: 100px 100px 100px;  
  grid-auto-rows: 50px;  
}
```

上面代码指定新增的行高统一为50px（原始的行高为100px）。



9 grid-template 属性, grid 属性

`grid-template` 属性是 `grid-template-columns`、`grid-template-rows` 和 `grid-template-areas` 这三个属性的合并简写形式。

`grid` 属性是 `grid-template-rows`、`grid-template-columns`、`grid-template-areas`、`grid-auto-rows`、`grid-auto-columns`、`grid-auto-flow` 这六个属性的合并简写形式。

从易读易写的角度考虑，**还是建议不要合并属性**，所以这里就不详细介绍这两个属性了。

项目属性

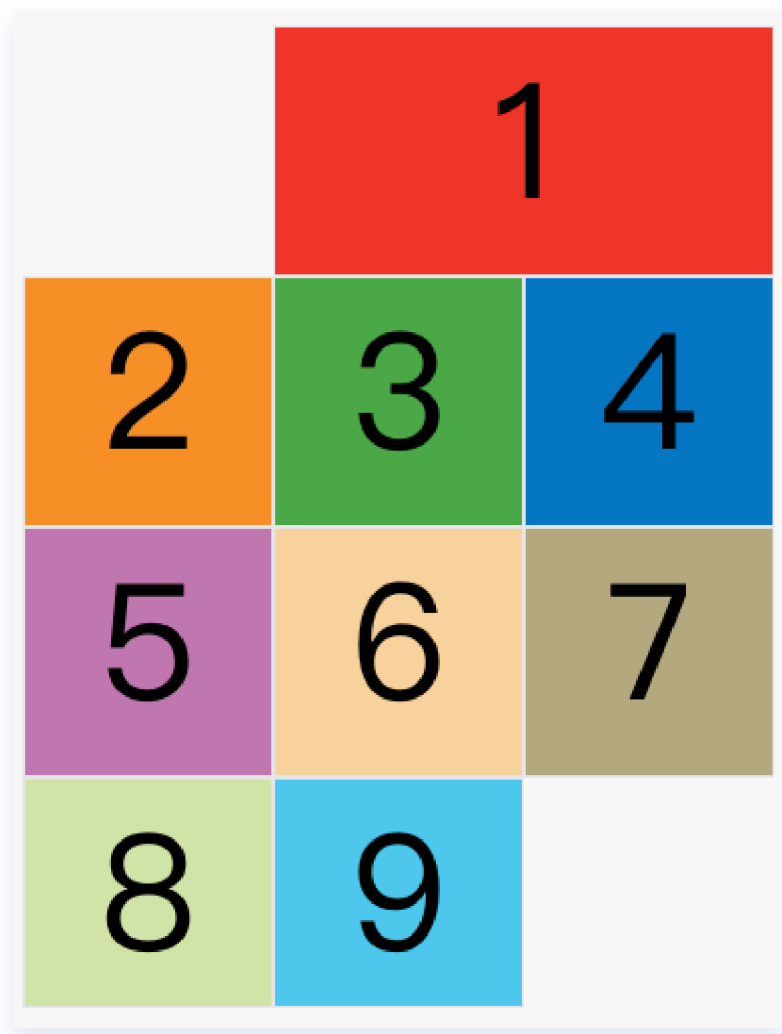
1 grid-column-start 属性, grid-column-end 属性, grid-row-start 属性, grid-row-end 属性

项目的位置是可以指定的，具体方法就是指定项目的四个边框，分别定位在哪根网格线。

- `grid-column-start` 属性：左边框所在的垂直网格线
- `grid-column-end` 属性：右边框所在的垂直网格线
- `grid-row-start` 属性：上边框所在的水平网格线
- `grid-row-end` 属性：下边框所在的水平网格线

```
.item-1 {  
  grid-column-start: 2;  
  grid-column-end: 4;  
}
```

上面代码指定，1号项目的左边框是第二根垂直网格线，右边框是第四根垂直网格线。



除了1号项目以外，其他项目都没有指定位置，由浏览器自动布局，这时它们的位置由容器的 `grid-auto-flow` 属性决定，这个属性的默认值是 `row`，因此会"先行后列"进行排列。

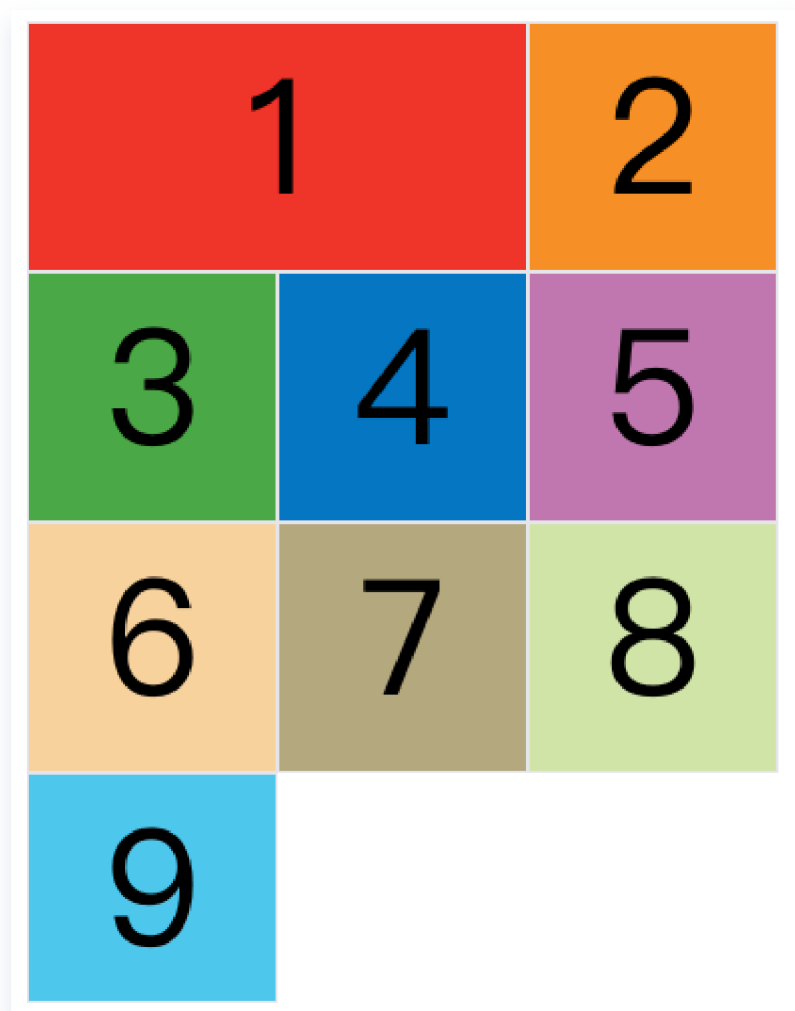
这四个属性的值，除了指定为第几个网格线，还可以指定为网格线的名字。

```
.item-1 {  
  grid-column-start: header-start;  
  grid-column-end: header-end;  
}
```

这四个属性的值还可以使用 `span` 关键字，表示"跨越"，即左右边框（上下边框）之间跨越多少个网格。

```
.item-1 {  
  grid-column-start: span 2;  
}
```

上面代码表示，1号项目的左边框距离右边框跨越2个网格。



使用这四个属性，如果产生了项目的重叠，则使用 `z-index` 属性指定项目的重叠顺序。

2 grid-column 属性, grid-row 属性

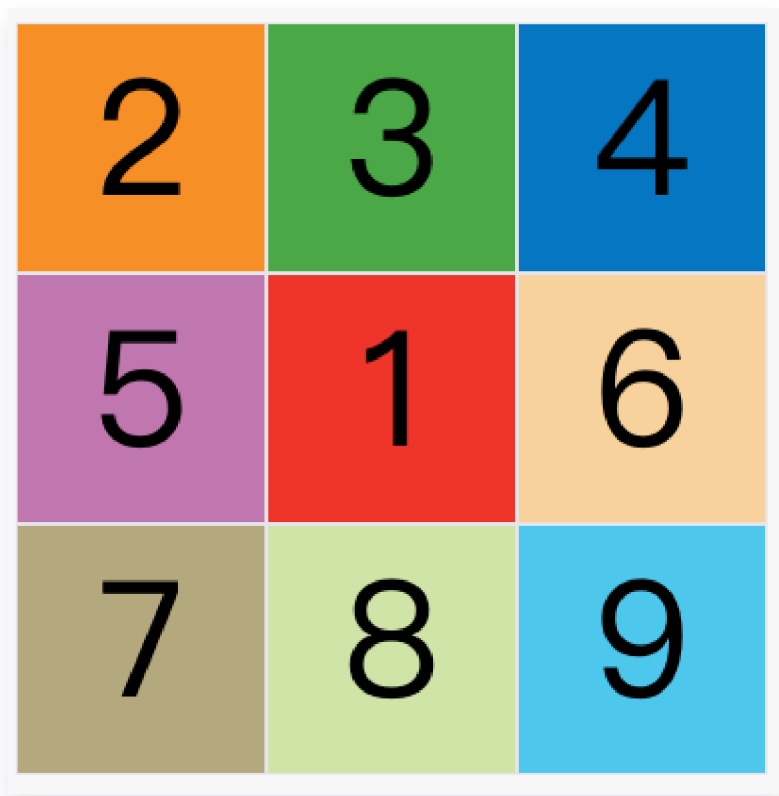
`grid-column` 属性是 `grid-column-start` 和 `grid-column-end` 的合并简写形式, `grid-row` 属性是 `grid-row-start` 属性和

3 grid-area 属性

`grid-area` 属性指定项目放在哪一个区域。

```
.item-1 {  
  grid-area: e;  
}
```

[上面代码](#)中, 1号项目位于 `e` 区域, 效果如下图。



`grid-area` 属性还可用作 `grid-row-start`、`grid-column-start`、`grid-row-end`、`grid-column-end` 的合并简写形式，直接指定项目的位置。

```
.item {  
  grid-area: <row-start> / <column-start> / <row-end> /  
  <column-end>;  
}
```

下面是一个例子。

```
.item-1 {  
  grid-area: 1 / 1 / 3 / 3;  
}
```

4 justify-self 属性, align-self 属性, place-self 属性

`justify-self` 属性设置单元格内容的水平位置（左中右），跟 `justify-items` 属性的用法完全一致，但只作用于单个项目。

`align-self` 属性设置单元格内容的垂直位置（上中下），跟 `align-items` 属性的用法完全一致，也是只作用于单个项目。

```
.item {  
  justify-self: start | end | center | stretch;  
  align-self: start | end | center | stretch;  
}
```

这两个属性都可以取下面四个值。

- start: 对齐单元格的起始边缘。
- end: 对齐单元格的结束边缘。
- center: 单元格内部居中。
- stretch: 拉伸, 占满单元格的整个宽度 (默认值)。

`place-self` 属性是 `align-self` 属性和 `justify-self` 属性的合并简写形式。

```
place-self: <align-self> <justify-self>;
```

下面是一个例子。

```
place-self: center center;
```

如果省略第二个值, `place-self` 属性会认为这两个值相等。

九.CSS单位的区别

1.像素（px）：

页面布局的基础，一个像素表示终端（电脑、手机、平板等）屏幕所能显示的最小的区域，像素分为两种类型：CSS像素和物理像素：

- CSS像素：为web开发者提供，在CSS中使用的一个抽象单位；
- 物理像素：只与设备的硬件密度有关，任何设备的物理像素都是固定的。

2.百分比（%）：

当浏览器的宽度或者高度发生变化时，通过百分比单位可以使得浏览器中的组件的宽和高随着浏览器的变化而变化，从而实现响应式的效果。一般认为子元素的百分比是相对于它的直接父元素的。

3.em和rem：

对于px更具灵活性，它们都是相对长度单位，它们之间的区别：em相对于自身或者父元素字体，rem相对于根元素（html字体大小：`document.documentElement.style.fontSize`）。

- em：文本相对长度单位。默认是相对父元素的font-size的倍数。但是如果元素本身已经设置了font-size，那么就会基于自身的字体大小进行计算
- rem：rem是CSS3新增的一个相对单位，相对于根元素（html元素，通常是16px）的font-size的倍数（即使自身已经设置了font-size）。作用：利用rem可以实现简单的响应式布局，可以利用html元素中字体的大小与屏幕间的比值来设置font-size的值，以此实现当屏幕分辨率变化时让元素也随之变化。

4.vw/vh:

与视口有关的单位，vw表示相对于视图窗口的宽度，vh表示相对于视图窗口高度，除了vw和vh外，还有vmin和vmax两个相关的单位。

- vw: 相对于视窗的宽度， $1\text{vw} = 1/100$ 视口宽度；
- vh: 相对于视窗的高度， $1\text{vh} = 1/100$ 视口高度；
- vmin: vw和vh中的较小值；
- vmax: vw和vh中的较大值。

十.display的属性值

1.display:none

none 是 CSS 1 就提出来的属性，将元素设置为none的时候既不会占据空间，也无法显示，相当于该元素不存在。

2.display:inline

它主要用来设置行内元素属性，设置了该属性之后设置高度、宽度都无效，

3.display:block

设置元素为块状元素，如果不指定宽高，默认会继承父元素的宽度，并且独占一行，即使宽度有剩余也会独占一行，高度一般以子元素撑开的高度为准，当然也可以自己设置宽度和高度。

4.display: list-item

此属性默认会把元素作为列表显示，要完全模仿列表的话还需要加上 `list-style-position`，`list-style-type`

5.display: inline-block

inline-block既具有block的宽高特性又具有inline的同行元素特性。

6.display: table:

table 此元素会作为块级表格来显示（类似table），它的子元素可以使用诸如 `display: table-row` 和 `display: table-cell`，以实现类似表格的多列布局。

同时我们在使用before和after双伪元素清除浮动那里也用到了这个display属性值

7.display: flex

设置弹性布局

十一.BFC

块级格式化上下文（Block Formatting Context, BFC）具有BFC特性的元素我们可以将其看作隔离的独立容器，容器内的元素不会在布局上影响到容器外的元素，而且BFC具有普通容器所没有的一些特性。

触发BFC的方法

- 浮动元素（`float: left | right`）；
- 绝对定位，固定定位（`position: absolute | fixed`）；
- flex（`display: flex`）；
- 'overflow' 特性不为 visible 的元素

注意, "display:table" 本身并不产生 "block formatting contexts"。但是, 它可以产生匿名框, 其中包含 "display:table-cell" 的框会产生块格式化上下文。总之, 对于 "display:table" 的元素, 产生块格式化上下文的是匿名框而不是 "display:table"

匿名框: 分为匿名块级框和匿名行内框, 如果如果一个块级框, 包含一个另一个块级框, 而这个块级框的兄弟元素没有被任何元素包裹, 那么就会为这个元素生成一个匿名块级框包含这个元素。

匿名行内框的道理也是一致的。

tips:

是的上层标签必须在一个

里面, 它不能单独使用, 相当于的属性标签。

表示一个表格, 表示这个表格中间的一个行 中间, 定义表头单元格。表格中的文字将以粗体显示, 在表格中也可以不用此标签, 标签内

BFC的布局规则

- BFC 就是一个块级元素, 块级元素会在垂直方向一个接一个的排列
- BFC 就是页面中的一个隔离的独立容器, 容器里的标签不会影响到外部标签
- 垂直方向的距离由margin决定, 属于同一个 BFC 的两个相邻的标签外边距会发生重叠, 但是不是同一个BFC基于不会发生重叠
- 计算 BFC 的高度时, 浮动元素也参与计算
- BFC可以使元素自适应其包含块的宽度, 防止元素溢出。

BFC解决的问题:

1.解决margin合并问题: 这个在之前说过

2.清除浮动带来的影响——父元素的高度塌陷问题:

方法：给父级元素添加 `overflow` 样式方法，触发 `BFC`

原理：计算BFC的高度时，浮动子元素也参与计算

3.清除浮动带来的影响——非浮动元素被浮动元素覆盖

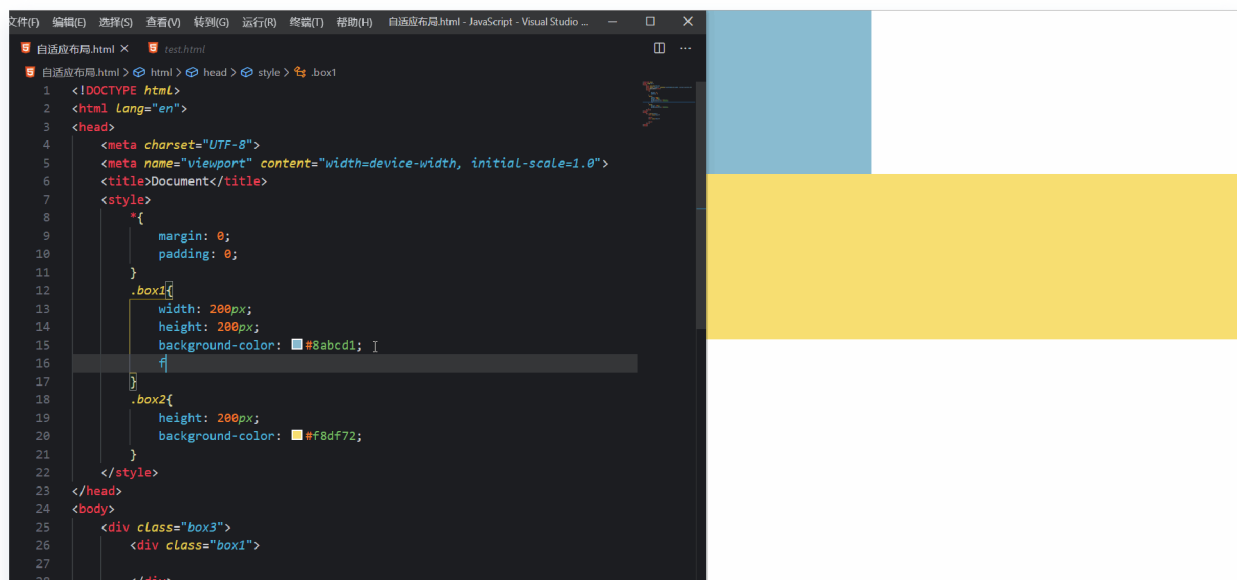
方法：给非浮动元素开启BFC

原理：BFC的区域不会与float box重叠

4.两栏自适应布局

方法：给固定栏设置固定宽度且使用浮动，给不固定栏触发BFC。

原理：BFC的区域不会与float box重叠



5.BFC可以使子元素自适应父元素的宽度和高度，防止元素溢出。

```
<style>
  .container {
    width: 300px;
    border: 1px solid black;
    height: 100px;
    overflow: hidden;
    /*如果不加这个子元素的内容过长会把父元素撑开*/
  }
```

```
.inner-content {  
  width: 100%;  
  background-color: lightblue;  
}  
</style>
```

十二. CSS 的布局方式:

1.静态布局

2.弹性布局 (flex)

3.浮动布局 (flow)

4.定位布局 (position)

5.网格布局 (Grid)

6.表格布局: table是一种常见的布局方式, 他可以将整个页面按照表格的方式设置为多行多列, 但是由于书写table标签比较麻烦尤其是涉及到table内嵌table的时候, 所以CSS给我们提供了 `display:table` 的方式可以让我们方便的使用table布局, 设置子元素为列的属性为 `display:table-cell`

十三.常用的CSS布局:

以下都是从子元素是块级元素的角度出发, 如果是行内元素将其设置为 `display:inline-block` 也是可用的, 特殊的情况会注明

1.水平居中

- 最简单的实现就是给子盒子设置 `margin:0 auto`，这个适用于块级元素，但是我们希望适用于行内元素，需要 `display:inline-block` 还需要给父元素设置 `text-align: center`
- 比较复杂的实现是使用 `position` 定位，子绝父相，然后给子元素给定 `left` 为 `50%`，之后再给定 `margin-left` 为负的自身宽度的一半（如果不关系宽度，可以直接用 `transform: translate(0, -50%)`）
- 还有一种方法是通过 `flex` 布局，给容器设置 `justify-content` 属性为 `center`，即主轴上的对齐方式为居中

2.垂直居中

- 使用 `position` 定位，子绝父相，然后给子元素给定 `top` 为 `50%`，之后再给定 `margin-top` 为负的自身宽度的一半，（如果不关心7宽度，可以直接用 `transform: translate(-50%)`）
- 设置绝对定位，配合 `top:0;bottom:0;`和 `margin:auto` 进行垂直居中
- 使用 `flex` 布局，给容器设置 `align-items` 属性为 `center`

首先我们要理解 `auto` 的意思是什么，`auto` 是 自动填充剩余空间

块级元素，即便我们设置了宽度，他还是自己占一行，如果我们给他的左右外边距设置为 `auto` 的时候，他会实现平分剩下的距离，从而达到一个水平居中的效果

但是块级元素的高度并不会自动扩充

但是我们可以通过 定位+margin 来实现

当我们给他定位让他上下都是 `0` 的时候，我们上下就有了多余空间，`auto` 就能平分剩余的空间去实现水平垂直居中

- 使用 `grid` 布局，但不设置容器上的属性，而是对子元素使用 `margin:auto`

这样可以一步到位实现水平垂直居中

- `table-cell`:

使用`table-cell`实现垂直居中,容器设置 `display: table-cell;vertical-align: middle;`

- 给容器加伪元素:

设置 `line-height` 等于容器的高度。给孩子设置 `display: inline-block;`

```
<div class="box">
  <div class="child">水平垂直居中</div>
</div>

<style>
.box {
  width: 200px;
  height: 200px;
  border: 1px solid;
  text-align: center;
}
.box::after {
  content: "";
  line-height: 200px;
}
.child {
  display: inline-block;
  background: red;
}

</style>
```

3.圣杯布局

什么是圣杯布局：

- header和footer各自占领屏幕所有宽度，高度固定。
- 中间的container是一个三栏布局。
- 三栏布局两侧宽度固定不变，中间部分自动填充整个区域。
- 中间部分的高度是三栏中最高的区域的高度。



怎么做？

- 先定义好header和footer的样式，使之横向撑满。
- 给外层的container设置 `padding-left: 200px; padding-right: 150px;`，给left和right空出位置
- 中间三栏左右设置固定宽度，中间设置宽度为百分之百，以便我们拖动页面改变页面宽度可以让中间的栏自适应
- 三列设为**浮动**，这个时候**footer清除浮动**（要不然container的浮动会遮盖住footer===》浮动的盒子会遮盖住处于标准文档流的元素）
- 这样因为浮动的关系，center会占据整个container，左右两块区域被挤下去了
- 接下来设置**左栏的** `margin-left: -100%;`，让left回到上一行
为什么？

简单的理解是margin-left设为负值，相当于盒子的大小减小了，减小了相当于父盒子宽度的100%，则左边的盒子就刚好遮住了center的左端

- 这时left并没有在最左侧，设置**相对定位**，然后通过 `left: -200px;` 把left拉回最左侧

- 对于right区域, 设置 margin-right: -150px; 把right拉回上一行, 也相当于自身宽度减小

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible"
content="ie=edge">
    <title>圣杯布局</title>
    <style type="text/css">
      body {
        min-width: 550px;
      }

      #header {
        background-color: #999999;
      }

      #middle{
        /* 2.把中间部分留出左右元素的宽度 */
        padding-left: 200px;
        padding-right: 150px;
      }

      #middle .column{
        float: left;
        height: 200px;
      }

      #left{
        width: 200px;
```

```

        background-color: #FFFF00;
        /* 4. 向左移动center的宽度 */
        margin-left: -100%;
        /* 5. 再向左移动自身宽度, 露出被覆盖的center块
*/

        position: relative;
        right: 200px;
    }

    #center{
        width: 100%;
        background-color: pink;
    }

    #right{
        /* 3.margin-right让右方元素覆盖自身, 达到消除
自身宽度的目的, 浮动到center上面去 */
        margin-right: -150px;
        width: 150px;
        background-color: #CCCCCC;
    }

    #footer {
        background-color: #999999;
    }

    .clearfix:after{
        display: table;
        content: '';
        clear: both;
    }
</style>
</head>
<body>
    <div id="header">header</div>
    <div id="middle" class="clearfix">

```

```
<div id="center" class="column">
  center
</div>
<div id="left" class="column">
  left
</div>
<div id="right" class="column">
  right
</div>
</div>
<div id="footer">footer</div>
</body>
</html>
```

还可以使用flex布局更为简单的实现：

- header和footer设置样式，横向撑满。
- container中的left、center、right依次排布即可
- 给container设置弹性布局 `display: flex;`
- left和right区域定宽，center设置 `flex: 1;` 即可

还有grid布局也可以，但是设置起来语义化程度也很差，因此我很少使用。

4. 双飞翼布局

相比圣杯布局，这是一种改进的做法

我们没有使用相对定位

- 先定义好header和footer的样式，使之横向撑满。
- 三栏均设置**浮动**

- 三列的左右两列分别定宽200px和150px，中间设置宽度为百分之百
- 设置中间的margin值为左右两个侧栏留出空间，margin值大小为left和right宽度
- 这样因为浮动的关系，center会占据整个container，左右两块区域被挤下去了
- 接下来设置left的 margin-left: -100%;，让left回到上一行最左侧，对于right区域，设置 margin-left: -150px; 把right拉回上一行（均设置了margin值）

无论是圣杯布局还是双飞翼布局，我们都要清除浮动带来的影响，最常见的做法是给三栏的父盒子container设置 `overflow:hidden` 属性（触发BFC）

5.CSS画三角形

1.border属性

使用 CSS 绘制三角形的第一种方法就是使用 `border` 属性。

给定一个宽度和高度都为 0 的元素，其 border 的任何值都会直接相交，我们可以利用这个交点来创建三角形

```
.triangle {  
    width: 0;  
    height: 0;  
    border: 100px solid;  
    border-color: orangered skyblue gold yellowgreen;  
}
```

如果我们只想要一个方向的三角形，可以让其它方向的border都设置为 `transparent`使其透明

2. linear-gradient

缺点：需要调试出合适的渐变角度

```
div {  
  width: 100px;  
  height: 100px;  
  background: linear-gradient(45deg, deeppink, deeppink  
50%, yellowgreen 50%);  
}
```

这是左下角和右上角两种不同颜色的三角形，如果只想要一个，可以让另一个颜色设置为透明：

```
div {  
  width: 100px;  
  height: 100px;  
  background: linear-gradient(45deg, deeppink, deeppink  
50%, transparent 50%);  
}
```

3.clip-path

使用 clip-path 可以为沿路径放置的每个点定义坐标。绘制三角形给多边形属性可以定义三个点： `top-left (0 0)`、`bottom-left (0% 100%)`、`right-center (100% 50%)`


```
.triangle{
    margin: 100px;
    width: 160px;
    height: 200px;
    background-color: skyblue;
    clip-path: polygon(0 0, 0% 100%, 100% 50%);
}
```

4.transform: rotate 配合 overflow: hidden 绘制三角形

```
.triangle {
    width: 141px;
    height: 100px;
    position: relative;
    overflow: hidden;

    &::before {
        content: "";
        position: absolute;
        top: 0;
        left: 0;
        right: 0;
        bottom: 0;
        background: deeppink;
        transform-origin: left bottom;
        transform: rotate(45deg);
    }
}
```

十四.媒体查询

在了解媒体查询前，有一些必要了解的知识：

响应式布局是什么呢？

响应式布局就是一个网站能够兼容多个终端，可以根据屏幕的大小自动调整页面的展示方式以及布局，我们不用为每一个终端做一个特定的版本！

响应式设计与自适应设计的区别？

响应式设计：响应式开发一套界面，通过检测视口分辨率，针对不同客户端在客户端做代码处理，来展现不同的布局和内容。

自适应设计：自适应需要开发多套界面，通过检测视口分辨率，来判断当前访问的设备是 PC 端、平板还是手机，从而请求服务端，返回不同的页面。

响应式设计与自适应设计如何选取？

- 页面不是太复杂的情况下，采用响应式布局的方式
- 页面中信息较多，布局较为复杂的情况，采用自适应布局的方式

响应式设计与自适应设计的优缺点？

响应式布局

优点：灵活性强，能够快速解决多设备显示适用问题

缺点

- 效率较低，兼容各设备工作量大
- 代码较为累赘，加载时间可能会加长

- 在一定程度上改变了网站原有的布局结构

自适应布局

优点

- 对网站复杂程度兼容更大
- 代码更高效
- 测试和运营都相对容易和精准

缺点： 同一个网站需要为不同的设备开发不同的页面，增加的开发的成本

媒体查询 (Media Queries)

Media Queries 能在不同的条件下使用不同的样式，使页面在不同终端设备下达到不同的渲染效果

Media Query 原理： 通过添加表达式来实现

媒体查询的语法非常简单，大概就是下面的样子：

```
@media 查询条件 {  
    CSS-Code;  
}  
  
/* 查询条件可以是媒体类型或是媒体特性 */  
/* 也可以是由and、not、only、or修饰组合而成的短语 */  
/* 多个条件也可以用 , (逗号) 连接, 相当于 or */  
/* 媒体特性需要用圆括号包围 */
```

媒体查询中的**媒体类型**主要是下面一些：

值	描述
all	适用于所有设备，默认值
print	适用于在打印预览模式下在屏幕上查看的分页材料和文档
screen	主要用于电脑屏幕，平板电脑，智能手机等
speech	主要用于屏幕阅读器等发声设备

媒体查询中的**媒体特性**非常多，这里列出一些常用的：

值	描述
orientation	portrait：网页可见区域高度大于等于宽度 landscape：网页可见区域宽度大于高度
aspect-ratio	网页可见区域宽高比
min-aspect-ratio	网页可见区域最小宽高比
max-aspect-ratio	网页可见区域最大宽高比
width	网页可见区域宽度值
min-width	网页可见区域最小宽度值
max-width	网页可见区域最大宽度值

十五.移动端响应式布局

解决方案一：媒体查询

```
/* iphone 5 */
@media screen and (max-width: 320px) {
  body {
    background-color: red;
  }
}
```

```
}  
/* iphone6 7 8 plus */  
@media screen and (min-width: 414px) {  
    body {  
        background-color: blue;  
    }  
}
```

缺点:

- 然使用媒体查询可以根据屏幕尺寸调整布局，但无法解决移动设备性能较低的问题。较大的页面和复杂的布局可能导致滚动卡顿、渲染延迟等性能问题，尤其是在低端设备上。
- 用户体验：响应式布局可能并不总能提供最佳的用户体验。因为设计是为桌面端而优化的，移动设备上的用户界面可能不够友好或难以操作。

解决方案二：百分比单位

通过百分比单位，可以使得浏览器中组件的宽和高随着浏览器的高度的变化而变化，从而实现响应式的效果。Bootstrap里面的栅格系统就是利用百分比来定义元素的宽高，CSS3 支持最大最小高，可以将百分比和 `max(min)` 一起结合使用来定义元素在不同设备下的宽高。

但是我们必须弄清楚css中子元素的百分比到底是相对谁的百分比。直接上结论吧：

子元素的 `height` 或 `width` 中使用百分比，是相对于子元素的直接父元素，`width` 相对于父元素的 `width`，`height` 相对于父元素的 `height`；子元素的 `top` 和 `bottom` 如果设置百分比，则相对于直接非 `static` 定位(默认定位)的父元素的高度，同样子元素的 `left` 和 `right` 如果设置百分比，则相对于直接非 `static` 定位(默认定位)的父元素的宽度；子元素的 `padding` 如果设置百分比，不论是垂直方

向或者是水平方向，都相对于直接父亲元素的 `width`，而与父元素的 `height` 无关。跟 `padding` 一样，`margin` 也是如此，子元素的 `margin` 如果设置成百分比，不论是垂直方向还是水平方向，都相对于直接父元素的 `width`；`border-radius` 不一样，如果设置 `border-radius` 为百分比，则是相对于自身的宽度，除了 `border-radius` 外，还有比如 `translate`、`background-size` 等都是相对于自身的；

这个方案的缺点：

- 计算困难，如果我们要定义一个元素的宽度和高度，按照设计稿，必须换算成百分比单位。
- 各个属性中如果使用百分比，相对父元素的属性并不是唯一的。造成我们使用百分比单位容易使布局问题变得复杂。

解决方案三：rem布局

`REM` 是 `CSS3` 新增的单位，并且移动端的支持度很高，Android2.x+,ios5+都支持。`rem` 单位都是相对于根元素html的 `font-size` 来决定大小的,根元素的 `font-size` 相当于提供了一个基准，当页面的size发生变化时，只需要改变 `font-size` 的值，那么以 `rem` 为固定单位的元素的大小也会发生响应的变化。因此，如果通过 `rem` 来实现响应式的布局，只需要根据视图容器的大小，动态的改变 `font-size` 即可

缺点：在响应式布局中，必须通过js来动态控制根元素 `font-size` 的大小，也就是说css样式和js代码有一定的耦合性，且必须将改变 `font-size` 的代码放在 `css` 样式之前

解决方案四：视口单位

单位	含义
vw	相对于视窗的宽度，1vw 等于视口宽度的1%，即视窗宽度是100vw
vh	相对于视窗的高度，1vh 等于视口高度的1%，即视窗高度是100vh
vmin	vw和vh中的较小值
vmax	vw和vh中的较大值

使用视口单位来实现响应式有两种做法：

1. 仅使用vw作为CSS单位

2. 搭配vw和rem

虽然采用 `vw` 适配后的页面效果很好，但是它是利用视口单位实现的布局，依赖视口大小而自动缩放，无论视口过大还是过小，它也随着时候过大或者过小，失去了最大最小宽度的限制，此时我们可以结合 `rem` 来实现布局

- 给根元素大小设置随着视口变化而变化的 `vw` 单位，这样就可以实现动态改变其大小
- 限制根元素字体大小的最大最小值，配合 `body` 加上最大宽度和最小宽度

解决方案五：图片响应式

这里的图片响应式包括两个方面，一个就是大小自适应，这样能够保证图片在不同的屏幕分辨率下出现压缩、拉伸的情况；一个就是根据不同的屏幕分辨率和设备像素比来尽可能选择高分辨率的图片，也就是当在小屏幕上不需要高清图或大图，这样我们用小图代替，就可以减少网络带宽了。

1.使用max-width（图片自适应）：

```
img {  
  display: inline-block;  
  max-width: 100%;  
  height: auto;  
}
```

`inline-block` 元素相对于它周围的内容以内联形式呈现，但与内联不同的是，这种情况下我们可以设置宽度和高度。`max-width` 保证了图片能够随着容器的进行等宽扩充（即保证所有图片最大显示为其自身的 100%。此时，如果包含图片的元素比图片固有宽度小，图片会缩放占满最大可用空间），而 `height` 为 `auto` 可以保证图片进行等比缩放而不至于失真

那么为什么不能用 `width: 100%` 呢？因为这条规则会导致它显示得跟它的容器一样宽。在容器比图片宽得多的情况下，图片会被无谓地拉伸。

2.使用srcset

```

```

在 `srcset` 中，可以为每个图像资源指定一个宽度描述符，表示该资源的实际像素宽度（或者像素密度比例）。浏览器会根据当前设备的像素密度和设备宽度，自动选择最佳的图像资源进行加载

在这个例子中，`srcset` 属性中的密度描述符分别为 `1x`、`2x` 和 `3x`，分别表示图像资源的像素密度比例为 1、2 和 3。浏览器会根据当前设备的像素密度自动选择合适的图像资源进行加载。

缺点：可能会有一些兼容性的问题，比如在Mac的Chorm中，可能会加载两张图片，一张是在srcset指定的，另一张实在src指定的

3.使用background-image

4.使用picture标签

`picture` 必须要写标签，否则无法显示，对 `picture` 的操作最后都是在上面

```
<picture>
  <source type="image/webp" srcset="banner.webp">
  
</picture>
```

使用 `source` 标签还可以对图片格式做一些兼容处理：

因为 `<source>` 元素可以使用 `srcset` 和 `type` 属性来指定不同尺寸和格式的图像资源，元素则作为最后的备选方案，用于在其它所有条件都不满足时显示的默认图像。

解决方案六：新的布局

- Flex弹性布局，兼容性较差
- Grid网格布局，兼容性较差

十六.less和scss/sass的基本使用

Sass(Scss)、Less 都是 **CSS 预处理器**，他们定义了一种新的语言，其基本思想是，用一种专门的编程语言为 CSS 增加了一些编程的特性，将 CSS 作为目标生成文件，然后开发者就只要使用这种语言进行 CSS 的编码工作。

为什么要使用 CSS 预处理器

- CSS 仅仅是一个标记语言，不可以自定义变量，不可以引用。
- 语法不够强大，比如无法嵌套书写，导致模块化开发中需要书写很多重复的选择器。
- 没有变量和合理的样式复用机制，使得逻辑上相关的属性值必须以字面量的形式重复输出，导致难以维护。

CSS 预处理器的的好处

- 提供 CSS 层缺失的样式层复用机制
- 减少冗余代码
- 提高样式代码的可维护性

CSS 预处理器的缺点

- 开发工作流中多了一个环节，调试也变得更麻烦。
- 预编译很容易造成后代选择器的滥用

何时使用 CSS 预处理器

- 系统级框架开发或者比较大型复杂的样式设计时
- 持续维护集成时
- 复用型组件开发时

Scss和Sass的语法和基本使用

Scss是Sass的新版本，它完全兼容普通的CSS语法，只是在原有的基础上添加了一些Sass的特性，使得它更容易被开发者接受和使用，而Sass的语法和与Ruby相似

不同	Sass	SCSS
文件扩展名	.sass	.scss
语法	与Ruby相似	与CSS相似
选择器语法	不用花括号、分号 (不能缩排)	使用花括号、分号

1. 文件扩展名：Sass文件的扩展名可以是 `.sass` 或者 `.scss`。`.sass` 使用缩进式语法，而 `.scss` 使用类似于CSS的语法。
2. 导入：使用 `@import` 指令将其他Sass文件导入到当前文件中。例如：`@import 'styles.scss';`
3. 变量：使用 `$` 符号定义一个变量，并在其他地方引用它。例如：

```
$primary-color: #007bff;  
.btn {  
  background-color: $primary-color;  
}
```

4. 嵌套：使用嵌套来简化样式规则的书写。可以在一个选择器内部嵌套另一个选择器，避免重复书写父选择器。例如：

```
.navbar {  
  background-color: #333;  
  ul {  
    list-style: none;  
    li {  
      display: inline-block;  
      margin-right: 10px;  
    }  
  }  
}
```

5. 混入 (Mixins)：使用混合来定义可重用的样式块。混合类似于函数，可以在需要的地方调用。例如：

```
@mixin button($background-color) {  
  background-color: $background-color;  
  color: white;  
  padding: 10px 20px;  
}  
  
.btn-primary {  
  @include button(#007bff);  
}
```

6. 继承：使用 `@extend` 指令继承一个选择器的属性。这可以减少重复的代码。例如：

```
.message {  
  border: 1px solid #ccc;  
  padding: 10px;  
}  
  
.error {  
  @extend .message;  
  color: red;  
}
```

7. 运算：Sass支持对数值进行简单的数学运算。可以使用加法、减法、乘法和除法等操作符。例如：

```
$width: 300px;  
.container {  
  width: $width + 100px;  
}
```

8. 导出：使用 `@export` 指令将Sass变量和混合导出为CSS自定义属性。这使得它们可以在其他文件中使用。例如：

```
$primary-color: #007bff;  
@export $primary-color;
```

Less的基本语法和使用

1. 文件扩展名：less文件的扩展名可以是 `.less`
2. 导入：使用 `@import` 指令将其他Sass文件导入到当前文件中。例如： `@import 'styles.scss';`
3. 变量：使用 `@` 符号定义一个变量，并在其他地方引用它。例如：

```
@primary-color: #007bff;  
.btn {  
  background-color: $primary-color;  
}
```

4. 嵌套：使用嵌套来简化样式规则的书写。可以在一个选择器内部嵌套另一个选择器，避免重复书写父选择器。例如：

```
.navbar {  
  background-color: #333;  
  ul {  
    list-style: none;  
    li {  
      display: inline-block;  
      margin-right: 10px;  
    }  
  }  
}
```

5. 混入（Mixins）：使用混合来定义可重用的样式块。混合类似于函数，可以在需要的地方调用。例如：

```
.rounded-corners {  
  border-radius: 5px;  
}  
  
.box {  
  color: #fff;  
  background-color: #007bff;  
  .rounded-corners; /* 使用混入 */  
}
```

```
.custom-button {  
  .rounded-corners; /* 使用混入 */  
  background-color: #ff6347;  
  border: 2px solid #ff6347;  
}
```

6. 继承: &:extend()

```
.button {  
  display: inline-block;  
  padding: 10px 20px;  
  border-radius: 5px;  
  background-color: #007bff;  
  color: #fff;  
}  
  
.submit-button {  
  &:extend(.button); /* 使用继承, 让.submit-button继  
承.button的样式 */  
  border: 2px solid #007bff;  
}  
  
.cancel-button {  
  &:extend(.button); /* 使用继承, 让.cancel-button继  
承.button的样式 */  
  background-color: #f44336;  
}
```

7. 运算: Sass支持对数值进行简单的数学运算。可以使用加法、减法、乘法和除法等操作符。例如:

```
@width: 300px;  
.container {  
  width: $width + 100px;  
}
```

十七.回流和重绘

回流(Reflow)

当 *Render Tree* 中DOM元素的尺寸、结构、或某些属性发生改变时,浏览器重新渲染部分或全部文档的过程称为回流。

会导致回流的操作:

- 页面首次渲染
- 浏览器窗口大小发生改变
- 元素尺寸或位置发生改变元素内容变化 (文字数量或图片大小等等)
- 元素字体大小变化
- 添加或者删除可见的 DOM 元素
- 激活 CSS 伪类 (例如: :hover)
- 查询某些属性或调用某些方法
- 一些常用且会导致回流的属性和方法:

`clientWidth`、`clientHeight`、`clientTop`、`clientLeft`

`offsetWidth`、`offsetHeight`、`offsetTop`、`offsetLeft`

`scrollWidth`、`scrollHeight`、`scrollTop`、`scrollLeft`

`scrollIntoView()`、`scrollIntoViewIfNeeded()`

`getComputedStyle()`

`getBoundingClientRect()`

`scrollTo()`

重绘(Repaint)

当页面中元素样式的改变并不影响它在文档流中的位置时（例如：`color`、`background-color`、`visibility` 等），浏览器会将新样式赋予给元素并重新绘制它，这个过程称为重绘。

性能影响

回流比重绘的代价要更高。

有时即使仅仅回流一个单一的元素，它的父元素以及任何跟随它的元素也会产生回流。现代浏览器会对频繁的回流或重绘操作进行优化：浏览器会维护一个队列，把所有引起回流和重绘的操作放入队列中，如果队列中的任务数量或者时间间隔达到一个阈值的，浏览器就会将队列清空，进行一次批处理，这样可以把多次回流和重绘变成一次。

如何避免

CSS

- 使用 CSS 动画而不是 JavaScript 实现的动画可以更好地利用浏览器的优化。CSS 动画通常可以通过 GPU 加速，并且使用 transition 动画避免回流和重绘。

Javascript

- 避免频繁操作样式
- 避免频繁操作 DOM, 创建一个 documentFragment, 在它上面应用所有 DOM 操作, 最后再把它添加到文档中。
- 也可以先为元素设置 display: none, 操作结束后再把它显示出来。因为在 display 属性为 none 的元素上进行的 DOM 操作不会引发回流和重绘。
- 对具有复杂动画的元素使用绝对定位, 使它脱离文档流, 否则会引起父元素及后续元素频繁回流。

十七.z-index压盖顺序

在浮动定位中，`z-index` 是用来定义层叠顺序的属性，它可以用于控制元素在网页上的显示顺序。

自定义压盖顺序：

- 属性值大的会压盖属性值小的，设置z-index属性的会压盖没有设置的。
- 如果属性值相同，比较HTML书写顺序，后写的压盖先写的。
- z-index属性只能设置给定位的元素才会生效，如果给没有定位的元素设置，不会生效。
- 从父现象。通常布局方案我们采用子绝父相，比较的是父元素的z-index值，哪个父元素的z-index值越大，表示子元素的层级越高。

解释：简单的说就是当父盒子设置了z-index值，那么子盒子在和它的元素去进行z-index来确定压盖顺序时，这个子盒子比较的是其父盒子的z-index

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <script
src="http://lib.sinaapp.com/js/jquery/2.0.2/jquery-
2.0.2.min.js"></script>
</head>
<style>
  .father{
    position: relative;
    background-color: #393636;
    width: 300px;
    height: 300px;
    z-index: -1;
  }
  .son{
    position: absolute;
    background-color: rgb(190, 38, 38);
    width: 100px;
    height: 100px;
    z-index: 999;
  }
  .passer{
    position: absolute;
    top: 50px;
    background-color: rgb(221, 120, 120);
    width: 100px;
    height: 100px;
    z-index: 2;
  }
</style>

<body>
  <div class="father">
    <div class="son">
```

```
    </div>
  </div>
  <div class="passer"></div>
</body>
</html>
```

上面设置了父盒子的z-index是-1，则无论子盒子的z-index多么大，都不会遮盖passer，而是passer遮盖子盒子。

十八.CSS动画

css中实现动画有两种方式：`transition` 过渡动画、`animation` 自定义动画。

1. transition 过渡动画

直接使用transition定义过渡动画时，它的各个参数定义的顺序是：定义顺序分别为：

- `transition-property`
- `transition-duration`
- `transition-timing-function`
- `transition-delay`

分别讲解这几个参数：

- `transition-property`：用于指定使用过渡效果的css属性，默认值为all，还可以设置width,height等等，当然也有不支持的CSS属性，比如文字内容`z-index`，`display`等等
- `transition-duration`：定义动画的过渡时间，默认值为`0s`，也就是说，如果不设置该属性，默认是没有过渡效果的。

- `transition-timing-function`: 定义动画事件函数, 该属性可以改变动画的执行速率以及轨迹。比如:

值	描述
<code>linear</code>	动画从头到尾的速度是相同的。
<code>ease</code> (缓解)	默认值 : 动画以低速开始, 然后加快, 在结束前变慢。
<code>ease-in</code>	动画以低速开始。
<code>ease-out</code>	动画以低速结束。
<code>ease-in-out</code>	动画以低速开始和结束。

- `transition-delay`: 用于设置动画延迟时间, 单位为 `s` 指的是过多长时间动画开始执行以及停止动画后过多久恢复原来的样子。

当然, 也可使用 `transition` 综合属性:

定义顺序分别为:

- `transition-property`
- `transition-duration`
- `transition-timing-function`
- `transition-delay`

```
transition: width 1s ease , height 1.5s linear 1s, color 2s ease-in-out ;
```

上面的代码与下面的代码的含义相同;

```
transition-property: width, height, color ;
transition-duration: 1s, 1.5s, 2s ;
transition-timing-function: ease, linear, ease-in-out ;
transition-delay: 0s, 1s, 0s ;
```

2.animation过渡动画

通过 `@keyframes` 自定义关键帧动画并为动画命名，可以在其中对每一帧进行设置。

使用自定义动画的元素，需要通过 `animation` 相关属性进行配置

- `animation-name`
- `animation-duration`
- `animation-timing-function` //用于定义时间函数: linear,ease...
- `animation-delay`
- `animation-iteration-count` //动画执行次数
- `animation-direction` //动画的执行方向
- `animation-fill-mode` //填充模式
- `animation-play-state` //后续有详细解释

我们还可在JavaScript中，通过驼峰式命名来访问/设置 `animation` 相关属性。

@keyframes

keyframes(关键帧)，在css样式表中可以通过 `@keyframes` 来设置关键帧动画，并指定动画名称供使用者锁定。

我们可以通过百分比来设置具体帧数的样式。

```
@keyframes animateName{
  0%   { width:50px; height:50px; }
  50%  { width:100px; height:100px; }
  100% { width:50px; height:50px; }
}
```

0% 和 100% 代表 首尾帧,也可分别使用 from、 to 替代

若某个元素想要使用对应名称的动画, 需要配置 animation-name: animateName 属性进行锁定。

比如:

```
<style>
  .box {
    width: 50px;
    height: 50px;
    background-color: pink;
    animation-name: test; /*类名为box的盒子绑定关键帧*/
  }
  @keyframes test { /*定义关键帧名字为test*/
    50% {
      width: 100px;
      height: 100px;
      border-radius: 50%;
      background-color: skyblue;
    }
  }
</style>
<body>
  <div class="box"></div>
</body>
```

animation-play-state

可设置动画的执行状态，通常通过JavaScript动态控制。

- 默认值为: `running`
- 设为 `paused` (暂停), 动画将停止执行。

```
<style>
  .box {
    width: 50px;
    height: 50px;
    background-color: pink;
    animation-name: test;
    animation-duration: 1s;
    animation-iteration-count: infinite;
  }
  @keyframes test {
    50% {
      width: 100px;
      height: 100px;
      border-radius: 50%;
      background-color: skyblue;
    }
  }
</style>
<body>
  <div class="box"></div>
  <button onclick="operation('running')">开始</button>
  <button onclick="operation('paused')">停止</button>
  // 回调函数传递参数使得动画暂停还是播放
  <script>
    function operation(mode) {

      document.querySelector("div").style.animationPlayState =
mode;
```



```
//用驼峰式命名的属性来设置CSS的animation-play-state
}
</script>
</body>
```

3.二者的区别和适用场景：

复杂的动画用animation。在遇到很复杂的动画那就用animation。因为animation可以定义关键帧，并且可以用js来控制，非常灵活。简单的动画效果用transition。

在性能方面：我们在用使用 animation 的时候这样可以改变很多属性，像我们改变了 width、height、position 等等这些改变文档流的属性的时候就会引起页面的回流和重绘，对性能影响比较大，但是我们用 transition 的时候一般会结合 transform 来进行旋转和缩放等不会引起页面的重排。

补充：渲染层合并是什么

渲染层合并（或称为图层合并）是一种优化技术，用于减少浏览器中的绘图操作，提高性能和渲染效率。在网页开发中，浏览器通常会将页面中的各个元素分别绘制到独立的图层上，然后再将这些图层合并到一个或多个复合图层上进行最终渲染。渲染层合并的过程中，浏览器会尽可能地减少图层的数量，以降低绘图操作的复杂度和性能消耗。

具体来说，对于每个图层，通常会有一个对应的图形上下文（GraphicsContext），用于处理该层的绘制操作。GraphicsContext 负责将绘制指令转换为位图，并将位图存储在共享内存中。这些位图最终会被上传到 GPU 中作为纹理，以便 GPU 进行后续的渲染操作。在 GPU 中，这些位图会被合成为最终的图像，并通过 GPU 进行绘制操作。绘制完成后，图像会被传输到显示器上，从而实现页面的展示。

十九.JS动画

怎么使用

- **使用 JavaScript 动画库 (JavaScript Animation Libraries)**：利用第三方 JavaScript 动画库，如 `anime.js`、`GSAP` (~~GreenSock Animation Platform~~) 等，这些库提供了丰富的 API 和功能，能够实现更复杂的动画效果，并提供更多的配置选项和性能优化。
- **使用 Canvas 绘图 (Canvas Drawing)**：通过 Canvas API 绘制图形，并在定时器或动画循环中更新图形的位置、大小等属性，从而创建自定义的动画效果。

js动画和CSS动画的区别

css动画：

- **简单易用**：CSS 动画可以通过简单的 CSS 属性和关键帧定义动画效果，不需要编写复杂的 JavaScript 代码。
- **性能优化**：浏览器会对 CSS 动画进行硬件加速，因此通常能够提供更流畅的动画效果，并且在性能方面表现较好。
- **适用于样式变化**：CSS 动画适用于对元素的样式属性进行动画，例如位置、大小、颜色等。
- **响应式设计**：CSS 动画可以通过媒体查询等技术轻松实现响应式设计，适应不同屏幕尺寸和设备。

JavaScript 动画：

- **灵活性**：JavaScript 动画提供了更高的灵活性和控制力，可以动态地根据各种条件和用户交互来调整动画效果。
- **复杂动画**：对于一些比较复杂的动画效果，如路径动画、物理动画等，JavaScript 动画通常更适合实现。

- **交互性**: JavaScript 动画能够更好地与用户交互和动态数据交互, 实现更复杂的动画效果和交互体验。

canvas绘图

Canvas 绘图是指使用 HTML5 中的 `<canvas>` 元素进行图形绘制和渲染的技术。Canvas 提供了一个**空白的画布**, 通过 JavaScript 脚本可以在上面绘制图形、文本和其他视觉元素, 从而实现各种动态和交互式的图形效果。

怎么使用?

使用 Canvas 进行绘图的基本步骤如下:

1. **创建 Canvas 元素**: 在 HTML 文件中, 使用 `<canvas>` 标签创建一个画布, 可以设置它的宽度和高度。

```
<canvas id="myCanvas" width="500" height="300"></canvas>
```

2. **获取 Canvas 上下文**: 使用 JavaScript 获取 Canvas 元素的上下文 (context), 通过上下文对象来进行绘图操作。

```
var canvas = document.getElementById('myCanvas');  
var ctx = canvas.getContext('2d');
```

3. **绘制图形**: 使用 Canvas 上下文提供的方法绘制各种图形, 如矩形、圆形、直线、路径等。

```
// 绘制矩形
ctx.fillStyle = 'red';
ctx.fillRect(50, 50, 100, 100);

// 绘制圆形
ctx.beginPath();
ctx.arc(250, 150, 50, 0, 2 * Math.PI);
ctx.fillStyle = 'blue';
ctx.fill();
```

4. **绘制文本**：可以使用 Canvas 上下文的 `fillText()` 或 `strokeText()` 方法在画布上绘制文本。

```
ctx.font = '20px Arial';
ctx.fillStyle = 'green';
ctx.fillText('Hello, Canvas!', 50, 250);
```

5. **添加事件处理**（可选）：如果需要交互式绘图，可以为 Canvas 元素添加事件处理程序，如鼠标点击、移动等事件。

```
canvas.addEventListener('click', function(event) {
    // 处理点击事件
});
```

6. **更新画布**：在进行绘制操作后，调用 `ctx.stroke()` 或 `ctx.fill()` 方法来将绘制的内容显示在画布上。
-

十九.before和after双冒号和单冒号有什么区别?

双冒号(::)和单冒号(:)都用于表示伪元素，但它们在语法上有一些区别。

- 双冒号(::)：在CSS3中引入了双冒号语法，用于表示伪元素。它是较新的语法规则，建议在使用CSS3伪元素时使用双冒号。例如：`::before`、`::after`。
- 单冒号(:)：在CSS2中引入了单冒号语法，最初用于表示伪类，如`:hover`、`:active`。然而，由于历史原因，单冒号也可以用于表示某些伪元素，如`:before`、`:after`。这种用法在CSS2中被允许，但在CSS3中不再推荐。

除了`::before`和`::after`之外，还有哪些常用的CSS3伪元素？

除了`::before`和`::after`，CSS3还引入了一些其他常用的伪元素。以下是其中几个常见的CSS3伪元素：

- `::first-line`：用于选中元素的第一行文本，可以对其应用特定的样式。
- `::first-letter`：用于选中元素的第一个字母，可以对其应用特定的样式。
- `::selection`：用于选中文本时的样式，例如文本的背景色和文本颜色等。
- `::placeholder`：用于设置表单元素的占位符文本的样式，允许自定义占位符文本的颜色、字体等。
- `::before`和`::after`之外的伪元素还可以通过`content`属性生成内容，例如`::marker`可用于自定义列表项标记的样式。

这只是一小部分常见的CSS3伪元素，CSS3还引入了其他伪元素，如`::nth-child`、`::last-child`、`::nth-of-type`等，用于选择特定的子元素或元素类型，并对其应用样式。

常见的单冒号(:)伪类有哪些?

单冒号(:)用于表示 CSS 中的伪类, 它们是一些用于选择特定状态或特定位置的元素的类别。以下是一些常见的单冒号伪类:

- `:hover`: 当鼠标悬停在元素上时应用的样式。
- `:active`: 当元素被激活或被点击时应用的样式。
- `:focus`: 当元素获得焦点时应用的样式, 通常在用户与表单元素进行交互时使用。
- `:visited`: 选择已访问过的链接的样式。
- `:first-child`: 选择父元素下的第一个子元素。
- `:last-child`: 选择父元素下的最后一个子元素。
- `:nth-child(n)`: 选择父元素下的第 n 个子元素。
- `:nth-of-type(n)`: 选择父元素下同类型元素中的第 n 个元素。
- `:not(selector)`: 选择不满足指定选择器的元素。
- `:empty`: 选择没有子元素或者没有文本内容的元素

二十.CSS3新特性

1.新增了一些选择器

不再只有单一的等号, 新增了 `$=xxx`(以xxx结尾), `^=` (以xxx开头) 等等, 初次还支持了一些针对子元素更加丰富的伪元素选择, 例如: `:nth-child`, `:last-child` 等等

```
ul li:nth-child(3){  
    color: #e08e8e;  
}
```

2.新增了一些样式:

新增三个边框属性：

- `border-radius`：创建圆角边框
- `box-shadow`：为元素添加阴影
- `border-image`：使用图片来绘制边框

新增了4个关于背景的属性，分别是 `background-clip`、`background-origin`、`background-size` 和 `background-break`

`background-clip`

用于确定背景画区，有以下几种可能的属性：

- `background-clip: border-box`; 背景从border开始显示
- `background-clip: padding-box`; 背景从padding开始显示
- `background-clip: content-box`; 背景从content区域开始显示
- `background-clip: no-clip`; 默认属性，等同于border-box

通常情况，背景都是覆盖整个元素的，利用这个属性可以设定背景颜色或图片的覆盖范围

`background-origin`

当我们设置背景图片时，图片是会以左上角对齐，但是是以 `border` 的左上角对齐还是以 `padding` 的左上角或者 `content` 的左上角对齐？`background-origin` 正是用来设置这个的

- `background-origin: border-box`; 从border开始计算background-position
- `background-origin: padding-box`; 从padding开始计算background-position
- `background-origin: content-box`; 从content开始计算background-position

默认情况是 `padding-box`，即以 `padding` 的左上角为原点

background-size

`background-size`属性常用来调整背景图片的大小，主要用于设定图片本身。有以下可能的属性：

- `background-size: contain;` 缩小图片以适合元素（维持像素长宽比）
- `background-size: cover;` 扩展元素以填补元素（维持像素长宽比）
- `background-size: 100px 100px;` 缩小图片至指定的大小
- `background-size: 50% 100%;` 缩小图片至指定的大小，百分比是相对包含元素的尺寸

background-break

元素可以被分成几个独立的盒子（如使内联元素span跨越多行），`background-break` 属性用来控制背景怎样在这些不同的盒子中显示

- `background-break: continuous;` 默认值。忽略盒之间的距离（也就是像元素没有分成多个盒子，依然是一个整体一样）
- `background-break: bounding-box;` 把盒之间的距离计算在内；
- `background-break: each-box;` 为每个盒子单独重绘背景

文字：

• word-wrap

语法： `word-wrap: normal/break-word`

- `normal`：使用浏览器默认的换行
- `break-all`：允许在单词内换行

#text-overflow

`text-overflow` 设置或检索当当前行超过指定容器的边界时如何显示，属性有两个值选择：

- `clip`：修剪文本

- `ellipsis`: 显示省略符号来代表被修剪的文本

#text-shadow

`text-shadow` 可向文本应用阴影。能够规定水平阴影、垂直阴影、模糊距离，以及阴影的颜色

#text-decoration

CSS3里面开始支持对文字的更深层次的渲染，具体有三个属性可供设置：

- `text-fill-color`: 设置文字内部填充颜色
- `text-stroke-color`: 设置文字边界填充颜色
- `text-stroke-width`: 设置文字边界宽度

#颜色

css3`新增了新的颜色表示方式`rgba`与`hsla`

- `rgba`分为两部分，`rgb`为颜色值，`a`为透明度
- `hsla`分为四部分，`h`为色相，`s`为饱和度，`l`为亮度，`a`为透明度

3.新增了过渡与动画

4. `flex` 弹性布局、`Grid` 栅格布局

5.媒体查询

二十一.CSS雪碧图的作用？

雪碧图即CSS Sprite，是把多个小图标合并在一张图片上，然后通过CSS背景定位来显示需要显示的图片部分。

作用：

1. **减少HTTP请求**：将多张图片合并成一张雪碧图后，只需要发起一次HTTP请求，而不是多次请求每张单独的图片，从而减少了网页加载所需的时间。
2. **加快网页加载速度**：减少HTTP请求可以显著提高网页加载速度，尤其是对于包含大量小图标或按钮的网页来说效果更加明显。
3. **减少带宽占用**：合并图片减少了每个HTTP请求的头部信息和其他开销，节约了服务器和客户端的带宽资源。
4. **精灵技术**：利用CSS background-position属性，可以在不同位置显示雪碧图中的不同部分，实现图标、按钮等元素的切换效果，提高了网页的交互体验。

二十二.display:none和visibility:hidden、opacity: 0的区别

结构

display:none: 浏览器不会渲染 display 属性为 none 的元素，不占据空间。

visibility: hidden: 元素被隐藏，但是会被渲染不会消失，占据空间。

opacity: 0: 透明度为 100%，元素隐藏，占据空间。

继承

display: none 不会被子元素继承

visibility: hidden: 会被子元素继承，子元素可以通过设置 visibility: visible; 来取消隐藏。

opacity: 0: 会被子元素继承,且，子元素并不能通过 opacity: 1 来取消隐藏。

性能

display none : 动态改变此属性时会引起重排, 性能较差。

visibility:hidden: 动态改变此属性时会引起重绘, 性能较高。

opacity: 0: 提升为合成层, 不会触发重绘, 性能较高。

事件监听

display none : 无法进行 DOM 事件监听。

visibility:hidden: 无法进行 DOM 事件监听。

opacity: 0: 可以进行 DOM 事件监听。

性能分析

display none : 会引起回流

visibility:hidden: 不会引起回流

opacity: 0: 不会引起回流

二十三: *opacity*和*rgba*区别

*rgba()*和*opacity*都能实现透明效果, 但最大的不同是*opacity*作用于元素, 以及元素内的所有内容的透明度, 而*rgba()*只作用于元素的颜色或其背景色, 也就是说设置*rgba*透明的元素的子元素是不会继承透明效果的。

在平时应用中, 如果需要元素背景, 以及元素里面的文字、子元素的文字等等有透明效果, 则用*opacity*属性, 只需要背景的透明色, 则对背景设置 *rgba*

二十四: line-height的继承问题

分为以下几种情况:

- 1.当父元素的line-height为具体带单位的数值的时候, 比如20px, 子元素没有设置line-height或设置为inherit则会直接继承父元素的line-height
- 2.当父元素的line-height为一个百分比时, 父元素的line-height的具体大小为父元素的font-size乘这个百分比, 子元素没有设置line-height或设置为inherit, 则会继承父元素计算完的line-height的结果
- 3.当父元素的line-height为一个数值但是没有单位时, 认为这个数值代表的是一个比例, 父元素的line-height的具体大小为父元素的font-size乘这个比例, 子元素没有设置line-height或设置为inherit, 则继承的是这个比例而不是计算的结果, 因此子元素的line-height就是自身的font-size去乘继承来的比例计算的结果

二十五.如何脱离文档流

目前常见的会影响元素脱离文档流的css属性有:

- ①float浮动。
- ②position的absolute和fixed定位。

二十六.CSS模块化

样式类名全局污染、命名混乱、样式覆盖等。这时, **css 模块化**就显得格外重要。

解决方法1: 使用less或者css modules, 两种都做到了css文件的模块化, 但是依赖于 webpack 构建和 css-loader 等 loader 处理。两者一般结合使用, 因为 less 没有提供像 CSS Modules 那样的作用域机制, 比如: `global(className)` 来声明一个全局类名。凡是这样声明的 class, 都不会被编译成哈希字符串, 以使得其它css模块可以引用。

但是我们一般情况下是混合使用

- 可以约定对于全局样式或者是公共组件样式, 可以用 `.css` 文件, 不需要做 `CSS Modules` 处理, 这样就不需要写 `:global` 等繁琐语法。
- 对于项目中开发的页面和业务组件, 统一用 `scss` 或者 `less` 等做 `CSS Module`

CSS Modules 可以配合 `classNames` 库实现更灵活的动态添加类名。

解决方法2: 放弃 `css`, `css in js` 用 js 对象方式写 `css`, 然后作为 `style` 方式赋予给 React 组件的 DOM 元素, 这种写法将不需要 `.css .less .scss` 等文件, 取而代之的是每一个组件都有一个写对应样式的 js 文件。

#

表示行中的一个列,需要嵌套在	
...	标签必须放在